Can Agent Memory Systems Track Evolving State?

Xinyi Fan*, Miri Liu*, Ruozhen Yang, Siru Ouyang, Jiawei Han

University of Illinois Urbana-Champaign
{xfan31, miri3, ruozhen2, siruo2, hanj}@illinois.edu

Abstract

As LLM-based agents are deployed for longer and higher-stakes tasks, their memory systems continue to have crucial gaps. While existing memory benchmarks focus largely on recall-shaped tasks, we argue an effective memory system must track the evolving state of the world; as facts, constraints, and decisions are revised over a long interaction, answers must reflect the current state and not a superseded one. We define this capability as state tracking and instantiate it in StateMemBench, a benchmark of 234 multi-session scenarios spanning two conversation-length regimes. Its closed-pool grading scores whether an answer reflects the current state, the superseded state, or fails otherwise, separating state-tracking failures from other errors by construction. Our analysis shows that this task is challenging for existing memory systems, retrieval-augmented baselines, and long-context baselines. We then present StateMem, a state-first memory method that explicitly tracks supersession and relational dependencies, and show it improves current-state accuracy over the strongest same-backbone baseline by 1.8× (0.205 → 0.363) on DeepSeek-V4-Flash and over the strongest memory system by 1.6× (0.149 → 0.233) on Qwen-3.5-9B, while remaining competitive with the long-context baselines. Finally, we show the same state approach can be applied as a lightweight single-call wrapper over existing memory systems, lifting current-state accuracy by +32 to +67 points on StateMemBench across six memory and retrieval backends. A length- and cost-matched control attributes +15 to +32 of those points to state structure rather than added context.

1 Introduction

Agents are now capable of (and deployed for) tasks that take place over multiple sessions, from managing end-to-end research pipelines (Gottweis et al. 2026; Lu et al. 2024) to autonomously working on software codebases (Wu 2024; Yang et al. 2024). The ability of an agent to keep working is, firstly, a crucial benchmark of its capabilities (Kwa et al. 2026), and, secondly, something that requires memory. Accordingly, many different strategies for managing agent memory have emerged, from extending the model's effective context (Ding et al. 2024) to external memory systems (Packer et al. 2024; Chhikara et al. 2025), just as many benchmarks have been proposed to evaluate these systems (Wu et al. 2025; Jiang et al. 2025; Maharana et al. 2024).

These systems and benchmarks focus heavily on optimizing recall of relevant facts, which is undoubtedly an important component of memory for humans and LLMs alike. However, we argue that as what we ask of agents becomes more time-consuming and complex, the ability of an agent to **track state** as it evolves is a crucial capability, and one that current systems largely overlook. For instance, as shown in Figure 1, we want agent memory systems to be able not only to recognize that the up-to-date number of annotations is 1200, but also to be able to recognize the state-level implications of that change.

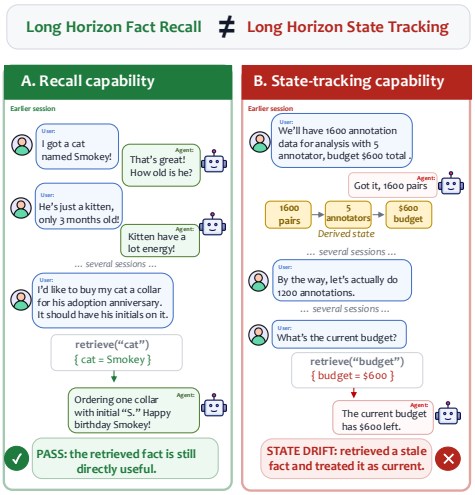

Figure 1: StateMemBench targets state tracking: maintaining currently operative values of facts, rules, and derived quantities under cross-session revision, separable from recall.

We call this failure **state drift**: the relevant fact is present in the assembled context, but the agent acts on a stale or incomplete version of it. The obstacle is not *retrieving* the fact, but rather *tracking* its relevance to current state, which we formalize in §3. While some recent benchmarks and memory systems (Wu et al. 2026a; Chao et al. 2026) have begun to center state as a first-class memory capability, and almost all systems update or revise memory in some sense, none cleanly isolates state tracking from the various other errors with which it co-occurs. We close this gap, and make four important contributions to agent memory:

*These authors contributed equally.

- We characterize **state tracking** as a capability distinct from recall, and give a labeling procedure that isolates state drift from other errors. Drift leads the failure distribution on several established memory benchmarks and persists even under perfect retrieval (§3; rates are judge-labeled upper bounds).
- We propose **StateMemBench**, a multi-domain benchmark targeted specifically at the problem of state tracking, with 234 multi-session scenarios spanning shorter and longer conversation lengths across three domains.
- We propose **StateMem**, a straightforward, state-first method, and show it improves over the strongest same-backbone baseline by 1.8× on DeepSeek (0.205 → 0.363) and over the strongest memory system by 1.6× on Qwen-9B (0.149 → 0.233), beating same-backbone long-context on both (0.149 on each).
- We show the state-tracking axis is **separable**: a lightweight answer-time wrapper form of StateMem improves every backend it is applied to, with a length- and cost-matched control isolating the share attributable to state structure rather than added context (§6.4).

2 Related Work

Long-term memory systems for LLM agents. Many existing memory systems optimize recall, including through virtual memory hierarchies with explicit read/write operations (Packer et al. 2024), reflection-based memory streams (Park et al. 2023), linked memory structures (Xu et al. 2025), or periodic summarization (Wu et al. 2026b). However, we consider recall to be orthogonal to state tracking. Even under perfect retrieval, a system still has to resolve which of the surfaced facts is currently valid (§3.2). Fewer systems attempt to address this. Zep (Rasmussen et al. 2025) attaches valid-time intervals to knowledge-graph edges and invalidates an edge on detecting a contradiction, which allows the system to track the validity of a single fact. The concurrent STALE (Chao et al. 2026) also targets state tracking and propagates updates, but does so through an LLM adjudicator over a predefined schema, whereas StateMem uses a deterministic, LLM-free pass over a dependency graph. Additionally, STALE constructs conflicts *implicitly*, so detecting them requires commonsense inference, while our evaluation testbed StateMemBench supplies conflicts *explicitly*, isolating state maintenance from that inference.

Dialogue state tracking. The term *state* has been extensively used in the task of **dialogue state tracking** (DST). Our work differs from this task in what *state* is and what is evaluated. DST prescribes the representation, whether as slot–value pairs over a domain ontology (Williams et al. 2013; Budzianowski et al. 2020), or its schema-guided (Rastogi et al. 2020) and open-vocabulary (Heck et al. 2020) relaxations, and evaluates that *representation* directly. On the other hand we do not require state to take any particular form; instead, state is what a memory system must maintain to answer correctly, and evaluation is purely behavioral. Additionally, DST corpora are cooperative, accumulating a user goal monotonically within one dialogue, and StateMemBench adversarially revises facts and user or agent decisions across multiple sessions to induce and measure stale-state failures (Kim et al. 2022; Bae et al. 2022).

Memory benchmarks for LLM agent memory. There are many memory benchmarks for LLM agent memory. Long conversational memory benchmarks (Maharana et al. 2024; Wu et al. 2025; Li et al. 2026) evaluate how well a model can answer questions about the user's messages, from surface-level factual recall to more complicated implicit intent. Some focus on personalizing responses to the user (Jiang et al. 2025), which overlaps with long conversational memory more generally. Similarly, user interaction benchmarks (Yao et al. 2024; Barres et al. 2025) evaluate tool-calling agents on their customer service skills, providing a controlled environment for measuring constraint adherence, but these are single-session, so agents do not necessarily need memory. Moving further from user dialogue, recent benchmarks (Zhao et al. 2026; He et al. 2026) evaluate how well an agent can use its memory system to accomplish tasks autonomously or semi-autonomously; notably, Wu et al. (2026a) evaluate whether agentic memory systems can learn through interaction how the environment changes, which they call dynamic state tracking. This targets learning the *environment's* dynamics as a world model, measured over trajectories of up to 115M tokens, thus entangling the task with large-scale retrieval. StateMemBench aims to isolate whether a memory system that *already holds* the relevant facts maintains the currently operative state, separately from retrieval and reasoning.

3 Problem Formulation: State Drift

Long-horizon agents fail even when the relevant information is in context. Precisely, **state drift** occurs when a fact, constraint, or dependency established or updated across prior context is present in the assembled context, but the agent acts on a stale or incomplete version of the state rather than the one in force at decision time. This separates failing to **maintain** state from failing to **acquire** it, as better retrieval does not fix the former.

3.1 Definition and Labeling

We consider drift in the context of possible neighboring failures. For instance, in a retrieval failure, the gold fact is not present, and in a schema mismatch, the answer is correct but badly formatted. For harder neighbors like reasoning failure, we consider tests like the following: the trace must demonstrably engage the current, up-to-date value, then err. Each $(case, turn, slot)$ failure is assigned to drift only once all other readings are excluded; unassignable points are dropped, not defaulted (*confirmed failures* = this filtered set). An adversarial filter plus cross-model-family judging (Appendix A) bias every exclusion *against* drift. We apply it on LongMemEval oracle (Wu et al. 2025), MemoryArena (MA) (He et al. 2026), and τ^2 (Barres et al. 2025).

Two judge passes agree on the binary drift decision at κ=0.67 on LongMemEval and 91.6% raw on MA-shopping (prevalence-deflated κ; PABAK=0.83); a cross-family judge (GPT-4o) agrees at κ=0.37 (Table 7). Two human annotators,

labeling under a structured protocol and, separately, describing failures blind to the taxonomy, recover the same categories, every blind description mapping to an existing bucket. Where humans depart from the judge they assign *less* drift, so judge rates are a ceiling within an already-conservative procedure (full agreement statistics in Appendix A).

Drift is also not a reasoning shortfall. On n=50 paired LongMemEval items, enabling DeepSeek-V4-Flash's reasoning trace does not help (84.0%→76.0%; McNemar p=0.34; Appendix A), while explicit state maintenance on the same task does (§6.4).

3.2 Drift Persists Under Perfect Retrieval

On LongMemEval oracle, recall is 1.0 by construction, so no residual failure can be retrieval in disguise. DeepSeek-V4-Flash fails 36 of 306 questions;[1] judges from two model families both confirm 44.4% (16/36; Wilson 95% CI [30%, 60%]) as state drift — the largest confirmed failure category under guaranteed evidence (Table 1). Drift notably concentrates where decisions integrate multiple sources; failures of multi-session questions drift at nearly three times the rate of knowledge-update failures (71.4% vs. 25.0%; 10/14 vs. 2/8, identical under either judge family). For details, see Appendix B.

3.3 Cross-Benchmark Prevalence

Drift is substantial but not universal across benchmarks (Table 1): it leads on MA-shopping (63.5%) and on τ^2-bench-Z (10/16 failures), but falls to 19.0% on MA-travel, where comprehension failure dominates (46.5%). Note that rates are LLM-judge labels and that human annotators tend to assign less drift (see Appendix A). However, this labeling cannot tell us why the wrong value wins. Judges can agree on whether a failure is drift, but not on its mechanism (such as a louder competitor or a genuine revision left unapplied) with mode κ near zero. To study the mechanism we must control it more explicitly. We therefore build StateMemBench, where each scenario's failure mechanism is fixed by construction and the superseded value is a scored outcome (Table 2 compares against existing benchmarks).

Table 1: **Cross-benchmark failure-mode distribution.** Each cell is the share of that benchmark's confirmed failures assigned to each bucket; the drift column is highlighted. †On LongMemEval oracle every gold fact is in the prompt by construction, so no failure there can be a retrieval failure. The small-n τ^2-bench-Z result (n=16) is reported in §3.3. Labeling, adjudication, and per-row details: Appendix A.

Benchmark	n	**drift**	retr.	comp.	schema	reason	FA
MemoryArena							
Shop.	200	**63.5**	22.5	2.5	0.0	9.5	–
Search	200	**38.5**	25.5	11.0	17.5	4.5	–
Travel	200	**19.0**	11.0	46.5	21.0	0.0	–
LongMemEval oracle							
LME†	36	**44.4**	0.0	0.0	25.0	2.8	27.8

4 StateMemBench

We release StateMemBench, a benchmark that aims to directly measure memory systems' ability to **track state.** We explain how failure modes are defined and computed, and we give an overview of the benchmark domains, the scenario generation pipeline, and our validation gates.

4.1 Failure Modes as Policy Divergence

We define failure modes mechanically rather than by hand-defining a taxonomy. We generate each scenario as a symbolic event program, or a sequence of typed state operations (rule declarations, value updates, scoped exceptions, commitments, and retractions) over ground, derived, and declared state. The correct answer to a probe is computed by replaying the program through a deterministic evaluator. Alongside the evaluator, we implement a family of lazy reader policies. These are small executable heuristics which each answer the probe according to some possible failure of a memory system. For instance, systems may trust the most recent mention, trust the most frequent value, return a stated value without recomputing it, or discard an anchored decision too eagerly. We get our trap scenarios exactly when one or more policies disagree with full replay. Those disagreeing policies then form the scenario's failure-mode signature. They instantiate five failure modes:

Status errors occur when a value changes (a tiered rule may move tiers, or a scoped override may lapse), but a reader remains anchored on the earlier, louder commitment and thus answers with the superseded value.

Salience errors occur when the valid value is present but a more frequently or more prominently mentioned competitor wins. Unlike a status error, nothing was actually superseded.

Sequence errors occur when a stated derived value is not recomputed after one of its inputs changes. The reader returns the stale derivation instead of carrying the update forward through the stated dependency.

Compound errors arise when several mechanisms interact.

Anti-trap scenarios test the opposite direction. An anchored answer will remain correct, and only an over-eager invalidation policy will result in divergence.

4.2 Domains

We instantiate the benchmark in three domains (research, shopping, and personal finance), chosen for three reasons: (1) shopping and personal finance capture single user–assistant state, while research captures collaborative project state; (2) each surfaces a unique mix of categorical choices and constraints; and (3) all three can be grounded in readily available public data. Note that public data supplies only the surface of a scenario, while state values are independently sampled, and no probe is answerable from the source material. For more information, see Appendix §D.5.

[1] The oracle pass covers LongMemEval's three cross-session question types (multi-session, temporal-reasoning, and knowledge-update) since single-session questions have no state to lose. The memory-system evaluation of §6.1 uses all 500 questions.

Table 2: Comparison with agent-memory and long-horizon benchmarks with state-tracking evaluation. (✓) = partial satisfied. *Scale* is reported in each benchmark's native unit. Column definitions and per-benchmark justifications are in Appendix C.

Benchmark	Multi-sess. dialogue	Scale (native unit)	Updates central	Super-session	Verifiable gold	Drift scored	Anti-update controls	Paired horizons
LoCoMo (2024)	✓	∼300 turns	✗	✗	✗	✗	✗	✗
LongMemEval (2025)	✓	∼40–80 sess. (S)	(✓)	(✓)	✗	✗	✗	✗
MemoryAgentBench (2025)	(✓)	varies	(✓)	(✓)	(✓)	✗	✗	✗
MemoryArena (2026)	(✓)	varies	✗	✗	✓	✗	✗	✗
STATE-Bench (2026)	✗	450 tasks	✗	✗	(✓)	✗	(✓)	✗
StateMemBench (ours)	✓	∼600 turns	✓	✓	✓	✓	✓	✓

4.3 Scenario Generation

We design each scenario so causal dependencies are stated explicitly (i.e., "Since we have 1600 annotators, we'll need 5 external annotators"), so a system is not required to infer that one fact bears on another. This isolates state tracking from the need to discover or assume relationships, letting us focus on whether a memory system actually maintains state as context evolves. The released benchmark has two length conditions: **Set A** (short) contains 190 single-probe scenarios of 18 sessions each (median 165 turns, ∼3k tokens), and **Set B** (long) contains 44 scenarios, each fusing three trap threads of different modes into one ∼38-session conversation (median 599 turns, 3.6× Set A in turns; ∼7–15k tokens) probed by a three-part question — **234 scenarios, 322 graded probes** in total. In Set B, each thread's gold-bearing events are buried among the *other* threads' live state, making Set B significantly more challenging. We detail scenario validation in Appendix §D.6.

Grounding and Rendering Each sampled program is grounded with surface material from the domain's real data (§4.2) and a strong LLM (`sonnet-4.6`) renders it into natural multi-session dialogue. The grounding supplies only surface vocabulary and never decides trap semantics, so it cannot contaminate a probe. We then programmatically verify that every load-bearing fact appears in its assigned session and that no banned phrase leaks, re-rendering any scenario that fails. Full pipeline details are in Appendix D.5.

Probe Design All scenarios carry a probe (the final turn, meant to test state tracking) and a gold truth (the correct response based on state changes). Probes are closed-pool, meaning that at scenario generation time we construct a set of *plausible* answers, though these are not shown at probe time. In addition to the gold truth, the pool contains the targeted policy's computed **drift answer**, which represents the value a reader that does not track state would anchor on, as well as relevant neutral distractors (3–4 options total). This means we can separate badly-tracked answers (*drift*) from totally-incorrect ones (*other*). The drift labels enable the error analysis in §6.3 over the entire benchmark.

5 StateMem

We introduce StateMem, a straightforward, state-first approach to memory that represents multi-turn conversational

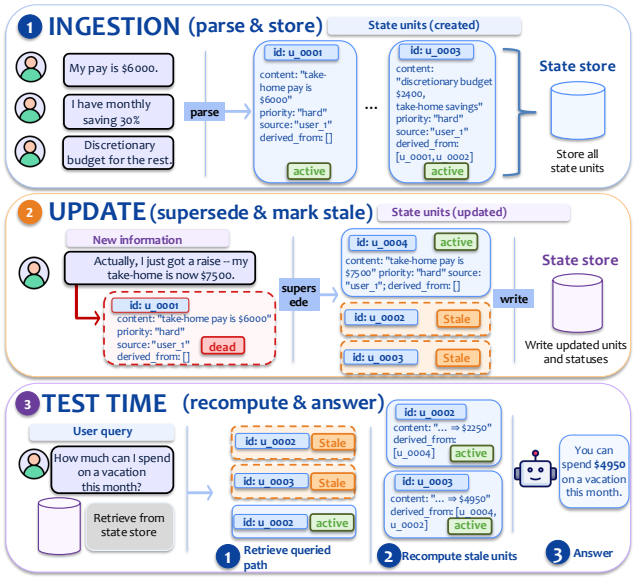

Figure 2: StateMem has three stages: an ingestion stage in which a conversation turn is parsed into state units, an update stage that updates the value of existing units, and a test time stage where the agent draws upon the valid state to answer the question. Here, we show how StateMem would handle a finance-based scenario.

memory as a collection of structured *state units*. As shown in Figure 2, StateMem operates in three stages: **ingestion** parses a turn into state units, **update** revises existing units, and **test time** uses the resulting coherent state to answer a question. Full prompts are in Appendix E.

5.1 Ingestion

A `TurnEncoder` applies one LLM call per conversational turn t, parsing it into zero or more state units, with the current compact rendering of all active units supplied as context. State stays compact at our scale, but the step is modular and can be swapped for slice-based retrieval of the `StateStore` as state grows. The encoder is domain-agnostic, admitting an optional `task_context` (Appendix E.1), though we report all results without it. Each state unit is a tuple $u = (id, content, priority, source, deps)$, with **id** a unique

identifier, **content** a free-form field, **priority** $\in$ {hard, soft}, **source** recording the originating turn and speaker, and **deps** a set of typed links (`derived_from` or `coupled_with`) to other units. Units persist across sessions in a `StateStore`. Where the data carries session dates, the encoder stamps each unit it creates with the turn's date; date handling is metadata-level throughout, so prompts are unchanged on undated data. While extracting, the `TurnEncoder` may also mark existing units as at risk of being **superseded** and volunteer replacements.

5.2 Update

The update stage performs two operations. First, the `StateStore` applies the flagged supersessions, setting the targeted unit's status to `superseded`, and any volunteered replacement is added as a new active unit. Second, a deterministic `Rechecker` traverses the dependency graph $G = (U, E)$, where an edge $(u_i, u_j) \in E$ records that u_i depends on u_j. For each u_i with an edge to a unit whose status *just* changed, the `Rechecker` sets u_i to `needs_recheck`, marking its content as possibly stale.

Note that a `superseded` unit stays in the store for auditability but is inactive and withheld from the next stage, while a `needs_recheck` unit stays active but flagged. Because dependencies are recorded at ingestion, the whole step runs in $O(|E|)$ and adds no LLM calls.

5.3 Test Time

The `StateStore` deterministically assembles the valid state from all active units, including those marked `needs_recheck`, and renders it as a structured block $\mathcal{S}$ grouped by priority and source, with each flagged unit shown alongside the trigger that flagged it. Dated units render with their dates, which double as recency markers, and where the benchmark supplies a question date it is included as a "today" anchor.

Answering costs one further LLM call, over $\mathcal{S}$, the question, and a prompt guiding recomputation of the current state (Appendix E.3). This deterministic layer decides which units are relevant and which are stale. The LLM only recomputes the answer given those flags.

6 Results

We show StateMem's competitive performance on existing memory benchmarks and evaluate StateMemBench on long-context LLMs, existing memory systems, retrieval baselines, and StateMem. We report the full results in Table 3; for a brief overview of each memory system evaluated and further implementation details, see §G.1. Each reported number is a single run per configuration: all answer and judge models decode at temperature 0 with reasoning traces disabled.

6.1 Existing Memory Benchmark Results

Among the systems we evaluate, StateMem scores highest of all *memory systems* on LongMemEval on both substrates (0.580 on Qwen-3.5-9B and 0.656 on DeepSeek-V4-Flash; Mem0 is next at 0.566/0.594), within a point of the long-context baseline on DeepSeek (0.666); and on LoCoMo

Table 3: State-agreement accuracy (gold rate) on StateMemBench: 190 in short scenarios ($\sim$165 turns, one probe each) and 132 in 44 long fused scenarios ($\sim$600 turns, three probes each), graded by a fixed deepseek-v4-pro judge. Memory systems (Mem0 (Chhikara et al. 2025), A-Mem (Xu et al. 2025), LightMem (Fang et al. 2026), MemoryOS (Kang et al. 2025), StateMem), retrieval baselines (BM25, text-embedding-3-small (OpenAI 2024)), and graph-RAG systems (nano-graphrag, HippoRAG, LightRAG, GraphRAG) run on each substrate (Qwen-3.5-9B (Qwen Team 2026), thinking off; deepseek-v4-flash (Xu et al. 2026), thinking off); long-context baselines are raw models with full history in context. StateMem ablations (indented) excluded from best-marking. Per column within each panel, best **bold**, second best underlined. ***StateMem's overall gold rate beats the strongest memory baseline on both backbones (paired McNemar test; both $p < 0.001$)

	Condition	Overall	Short	Long
Long-context	Qwen-3.5-9B	0.149	0.137	0.167
	Qwen-3.6-35B-A3B	0.097	0.063	0.146
	GPT-5.4-Nano	**0.277**	**0.311**	**0.227**
	DeepSeek-V4-Flash	0.149	0.132	0.174
Qwen-3.5-9B	No memory	0.006	0.000	0.015
	BM25	0.125	0.095	0.167
	Dense	0.130	0.105	0.167
	nano-graphrag	0.143	0.126	0.167
	HippoRAG	0.130	0.105	0.167
	LightRAG	0.115	0.089	0.152
	GraphRAG	0.224	0.216	**0.235**
	Mem0	0.149	0.116	0.197
	A-Mem	0.127	0.100	0.167
	LightMem	0.019	0.021	0.015
	MemoryOS	0.025	0.026	0.023
	StateMem (ours)	**0.233*****	**0.237**	0.227
	– *extraction-only*	0.155	0.126	0.197
	– + *supersession*	0.171	0.137	0.220
	– *w/o dep.-propagation*	0.222	0.202	0.250
	– *w/o recompute-guid.*	0.208	0.195	0.227
deepseek-v4-flash	No memory	0.003	0.000	0.008
	BM25	0.143	0.105	0.197
	Dense	0.205	0.168	0.258
	nano-graphrag	0.183	0.174	0.197
	HippoRAG	0.184	0.132	0.258
	LightRAG	0.093	0.058	0.144
	GraphRAG	0.174	0.137	0.227
	Mem0	0.177	0.168	0.189
	A-Mem	0.199	0.158	0.258
	LightMem	0.012	0.021	0.000
	MemoryOS	0.025	0.011	0.045
	StateMem (ours)	**0.363*****	**0.411**	**0.295**
	– *extraction-only*	0.174	0.153	0.205
	– + *supersession*	0.298	0.337	0.242
	– *w/o dep.-propagation*	0.373	0.400	0.333
	– *w/o recompute-guid.*	0.301	0.326	0.265

(0.566/0.592) it leads all memory systems and is level with long-context on DeepSeek (0.592 vs. 0.587) (Table 4, with full version including question types in Table 11).

The per-question-type numbers indicate where the gain comes from. Both benchmarks include question types that require tracking an update, and StateMem's margins are concentrated on those: temporal reasoning on LongMemEval (0.624 vs. 0.391 for long-context on deepseek, 0.398 vs. 0.248 on Qwen) and knowledge update (0.795 on deepseek). On single-hop recall and aggregation questions, where having the full history in context is hard to beat, StateMem comes within a few points of long-context while reading only a bounded state. We therefore conclude state tracking does not cost recall.

6.2 StateMemBench Results

On StateMemBench, long-context ceases to be the strong baseline it is on recall benchmarks: even the best long-context model (GPT-5.4-Nano) reaches only 0.277 overall, and same-backbone long-context scores just 0.149 on both DeepSeek-V4-Flash and Qwen-3.5-9B. The adversarial traps defeat simply holding the full transcript in context.

Within each backbone panel, StateMem is the strongest condition. On DeepSeek it reaches 0.363, beating same-backbone long-context by 2.4× (vs. 0.149), the best memory system by 1.8× (A-Mem, 0.199), and the best baseline of *any* type by a similar factor (Dense retrieval, 0.205). On Qwen, StateMem (0.233) leads the best memory system by 1.6× (Mem0, 0.149) and beats long-context (0.149), with GraphRAG (0.224) statistically level (McNemar over the 322 shared probes: 38 vs. 35 discordant, p=0.82). Explicit state tracking thus separates decisively from graph-structured retrieval on the stronger backbone, where the drift-rate itself falls (−15 pp, §6.3); on the weaker, the gain comes mainly from converting off-pool non-answers into in-pool answers. This is consistent with §6.3's finding that a capable answerer is needed to convert an explicit state representation into correct answers. We analyze *why* the memory baselines underperform (as well as why StateMem largely does not) in §6.3.

Category ordering holds across substrates. Anti-trap sits near ceiling almost everywhere (StateMem ≈0.87 on DeepSeek), since its gold answer is the anchored value. The compositional modes separate the systems—baselines are near zero on salience, sequence, and compound, and StateMem recovers most of the last two. Salience appears to be a substrate limit rather than a memory-design one, since every Qwen arm scores ≤ 0.18 against 0.569 for GPT-5.4-Nano.

Which components pay their way? Extraction alone recovers little (0.174 / 0.155); supersession marking is the largest single step (0.174→0.298 on DeepSeek), and dropping recompute guidance costs 6 pp. On the other hand, dependency propagation helps its target modes but over-propagates on Set B anti-traps (−12.5 pp), and removing it leaves DeepSeek slightly better (0.373 vs. 0.363). Again, since StateMem mirrors the policy family behind the traps (§4.1), we read these margins as an upper bound and §6.1 as a test of its generalization.

6.3 Why Do Memory Systems Drift?

We utilize the closed-pool design discussed in §4.3 to better understand the state tracking behavior of the evaluated models. We calculate **drift-rate**, or how often an incorrect answer was the selected drift target (drift / n), where a lower rate is better, and report those results in Table 5. Reading it requires separating two modes of failure: (1) when the method is actively anchoring on a stale value (an in-pool drift); and (2) when the method fails to surface a recognizable or plausible option at all (off-pool). Methods that answer predominantly off-pool (no memory, LightMem, and MemoryOS, which land off-pool on 60–94% of cells) post deceptively low drift-rates by construction, not because they resist the drift target trap but because they rarely produce an in-pool (plausible) answer. We therefore report in-pool count (I-P) alongside drift-rate, and give full closed-pool breakdowns in Appendix Tables 14 and 15.

Table 4: Accuracy on LongMemEval (full set, n=500, k=20) and LoCoMo (n=1,985), with each method run on both substrates (Qwen-3.5-9B; deepseek-v4-flash; thinking off) and all answers graded by a fixed deepseek-v4-pro judge. Per column, best in **bold**, second best underlined.

	LongMemEval		**LoCoMo**	
Condition	Qwen-9B	DS-V4	Qwen-9B	DS-V4
Long context	0.550	**0.666**	**0.612**	0.587
No memory	0.062	0.056	0.010	0.005
Mem0	0.566	0.594	0.481	0.462
A-Mem	0.520	0.510	0.266	0.279
LightMem	0.064	0.056	0.027	0.020
MemoryOS	0.080	0.072	0.040	0.031
StateMem (ours)	**0.580**	0.656	0.566	**0.592**

Among methods that do engage the closed pool, the weak backbone (Qwen-3.5-9B) drifts heavily and near-uniformly: drift-rate sits at 61–66% for long-context, Mem0, BM25, and StateMem alike; so on a weak answerer the memory layer barely changes the outcome. The methods separate on the stronger backbone (DeepSeek-V4-Flash). StateMem's drift-rate falls the most, from 64.3% to 49.1% (−15 pp), and its correct-answer count rises the most, from 75 to 117 (+42): a capable answerer **converts would-be drifts into gold** once the state representation makes the operative value explicit. Long-context, Mem0, and BM25 barely move (drift-rate shifts by 1–3 pp, correct answers by 0–9), indicating their bottleneck is **the memory layer, not the answerer**. StateMem is also the most engaged answerer on both backbones, with the fewest off-pool answers (32 on Qwen, 36 on DeepSeek) and the highest in-pool count, indicating the accuracy is not due to higher abstention.

6.4 StateMem as a Wrapper

To test the claim that state tracking is a separable axis, we build StateMemWrapper: StateMem's approach to state as a prompt-level transformation of the answer call, with none

Table 5: Closed-pool (n=322) drift-rate and in-pool engagement, methods that produce non-trivial in-pool counts on at least one backbone. *In-pool* (I-P) = method produced one of the closed-pool options. *Drift-rate* (D-R) = drift / n (errors are 0 for every method). StateMem's I-P is **bold**.

	Qwen-3.5-9B		DeepSeek-V4-Fl.	
Method	I-P	D-R (↓)	I-P	D-R (↓)
full context	263	65.5	258	64.0
Mem0	254	62.7	268	59.9
BM25	243	60.9	244	59.9
LightMem	109	32.0	52	14.9
StateMem (ours)	**290**	64.3	**286**	49.1

of its machinery. The wrapper receives the transcript, the backend's retrieved chunks, and the question, and replaces the answer call the backend would have made anyway, so it adds no LLM calls.

The call fuses two stages. **Trace** writes, scoped by question, the value chain of every slot the question touches—initial value, each revision, and current operative value, in chronological order with turn numbers—plus the latest stated inputs of every standing rule. **Resolve** then commits under four precedence rules targeting the failure modes of §3.3: later supersedes earlier, standing rules outrank instances, derived quantities are recomputed rather than quoted, and a fact is retired only by explicit supersession or expiry. The trace is capped at 250 words and committed before the answer, which blocks post-hoc justification. Overhead over the matched control is a fixed 155-token instruction and that $\leq$250-word trace, both inside the one call (Appendix F.5).

The control, WRAPPER-CTRL, matches the wrapper on call count, word budget, answer format, and context; its first stage is generic question-conditioned extraction, with no chains, no turn numbers, and no resolution rules. Δ(wrapper $-$ wrapper-ctrl) therefore isolates structure from sheer text volume. Both prompts appear verbatim in Appendix F.1.

Evaluation We evaluate all six backends on both substrates against frozen paired n=60 sets from STATEMEMBENCH and LongMemEval (Wu et al. 2025) (Table 6). Each store is built once and answered three ways, so *alone*, *+Ctrl*, and *+SMW* face identical retrieved chunks under a single fixed judge.

The wrapper improves every backend on both benchmarks. On STATEMEMBENCH it adds $+31.7$ to $+66.6$ points over each backend alone (largest for the near-floor LightMem and MemOS), of which structure contributes $+15.0$ to $+31.7$. This is significant in all twelve cells (paired McNemar $p\leq 0.04$, bootstrap CIs exclude zero) and exceeds the control in 23 of 24. On LongMemEval the split is substrate-dependent, as structure carries the gain on Qwen ($+11.7$ to $+25.0$; the control alone often hurts), while on the DeepSeek substrate the control recovers most of the lift and structure adds only -5 to $+5$. This implies a strong answerer has less need for the state structure. Additionally, a question-blind 250-word summary falls below the no-wrapper baseline on both benchmarks and carries the highest drift rate of any arm, showing the importance of question conditioning.

Table 6: **StateMemWrapper composition sweep**; each backend ingests with its unmodified pipeline and is evaluated on the same frozen paired n=60 set per benchmark (StateMemBench: 30 short + 30 long scenarios, k=10; LongMemEval: k=20) under three answer conditions: the backend *alone*, a length- and cost-matched generic control (*+Ctrl*), and our state-tracing wrapper (*+SMW*). All answers are scored by a fixed deepseek-v4-pro judge. Best condition per row in **bold**. Dense on LongMemEval/Qwen has n=59 (one case failed at ingest).

		Qwen-3.5-9B			deepseek-v4-flash		
	Backend	Alone	+Ctrl	+SMW	Alone	+Ctrl	+SMW
StateMemBench	Mem0	25.0	28.3	**56.7**	28.3	56.7	**71.7**
	A-Mem	23.3	35.0	**56.7**	33.3	51.7	**75.0**
	LightMem	3.3	30.0	**61.7**	1.7	51.7	**68.3**
	MemOS	5.0	25.0	**56.7**	1.7	50.0	**68.3**
	BM25	20.0	31.7	**56.7**	21.7	48.3	**70.0**
	Dense	21.7	28.3	**56.7**	31.7	43.3	**70.0**
LongMemEval	Mem0	38.3	36.7	**48.3**	45.0	58.3	**60.0**
	A-Mem	41.7	31.7	**56.7**	35.0	51.7	**55.0**
	LightMem	5.0	11.7	**28.3**	3.3	31.7	**35.0**
	MemOS	6.7	8.3	**23.3**	6.7	31.7	**36.7**
	BM25	16.7	16.7	**36.7**	16.7	40.0	**41.7**
	Dense	47.5	32.2	**50.8**	53.3	**61.7**	56.7

Notably, the wrapper outperforms STATEMEM itself on the same scenarios (0.283 vs. $\geq$0.567 on Qwen). When the transcript fits in context, resolving state at answer time is good enough, but the persistent store can be useful when it does not fit. Prompts, backend protocol, blind-summary ablation, and cost analysis are in Appendix F; per-cell tables are Tables 12 and 13.

7 Conclusion

We analyze state tracking as a capability distinct from recall and reasoning, and show that state drift is already present, though unmeasured, in the failure states of existing benchmarks even under perfect retrieval. We introduce STATEMEMBENCH, which evaluates state tracking through 234 multi-session scenarios spanning five failure modes (status, salience, sequence, compound, and anti-trap) across three domains, and find that existing memory systems and long-context baselines struggle across all of them. We then present STATEMEM, a state-first method representing state and its relational dependencies, which improves over the strongest same-backbone baseline by up to 1.8$\times$ on STATEMEMBENCH, surpasses same-backbone long-context there, and stays competitive with it on recall benchmarks; ablations attribute the gain chiefly to supersession marking and recompute prompting. Finally, the state-tracking axis is separable: applied as an answer-time wrapper, it lifts every backend we test, with a matched control isolating the structural share. Current memory systems lack just this explicit state discipline.

Acknowledgments

Research was supported in part by the AI Institute for Molecular Discovery, Synthetic Strategy, and Manufacturing: Molecule Maker Lab Institute (MMLI), funded by U.S. National Science Foundation under Award 2505932, NSF IIS 25-37827, and the Institute for Geospatial Understanding through an Integrative Discovery Environment (I-GUIDE) by NSF under Award No. 2118329. The research has used the Delta/DeltaAI advanced computing and data resource, supported in part by the University of Illinois Urbana-Champaign and through allocation #250851 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program, which is supported by National Science Foundation grants OAC 2320345, #2138259, #2138286, #2138307, #2137603, and #2138296. Any opinions, findings, and conclusions or recommendations expressed herein are those of the authors and do not necessarily represent the views, either expressed or implied, of DARPA or the U.S. Government. Miri Liu was supported by the Amazon AI PhD Fellowship.

References

Bae, S.; Kwak, D.; Kang, S.; Lee, M. Y.; Kim, S.; Jeong, Y.; Kim, H.; Lee, S.-W.; Park, W.; and Sung, N. 2022. Keep Me Updated! Memory Management in Long-term Conversations.

Barres, V.; Dong, H.; Ray, S.; Si, X.; and Narasimhan, K. 2025. τ^2-Bench: Evaluating Conversational Agents in a Dual-Control Environment.

Budzianowski, P.; Wen, T.-H.; Tseng, B.-H.; Casanueva, I.; Ultes, S.; Ramadan, O.; and Gašić, M. 2020. MultiWOZ – A Large-Scale Multi-Domain Wizard-of-Oz Dataset for Task-Oriented Dialogue Modelling.

Chao, H.; Bai, Y.; Sheng, R.; Li, T.; and Sun, Y. 2026. STALE: Can LLM Agents Know When Their Memories Are No Longer Valid?

Chhikara, P.; Khant, D.; Aryan, S.; Singh, T.; and Yadav, D. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413.

Ding, Y.; Zhang, L. L.; Zhang, C.; Xu, Y.; Shang, N.; Xu, J.; Yang, F.; and Yang, M. 2024. LongRoPE: Extending LLM Context Window Beyond 2 Million Tokens.

Fang, J.; Deng, X.; Xu, H.; Jiang, Z.; Tang, Y.; Xu, Z.; Deng, S.; Yao, Y.; Wang, M.; Qiao, S.; Chen, H.; and Zhang, N. 2026. LightMem: Lightweight and Efficient Memory-Augmented Generation.

Gottweis, J.; Weng, W.-H.; Daryin, A.; Tu, T.; Sirkovic, P.; Myaskovsky, A.; Glowaty, G.; Weissenberger, F.; Orlandi, A.; Popovici, D.; Palepu, A.; Rong, K.; Tanno, R.; Saab, K.; Zhang, F.; Blum, J.; Carroll, A.; Kulkarni, K.; Tomašev, N.; Zverinski, D.; Rendulic, I.; Vedadi, E.; Hasler, F.; Rimanic, L.; Boia, M.; Budiselic, I.; Feinstein, B.; Bellaiche, M.; Sheffer, T.; Freyberg, J.; Ratcliff, J.; Bertolli, O.; Chou, K.; Hassidim, A.; Gokturk, B.; Vahdat, A.; Guan, Y.; Dhillon, V.; Vaishnav, E. D.; Lee, B.; Costa, T. R. D.; Penadés, J. R.; Peltz, G.; Matias, Y.; Manyika, J.; Hassabis, D.; Xu, Y.; Kohli, P.; Pawlosky, A.; Karthikesalingam, A.; and Natarajan, V. 2026. Accelerating scientific discovery with Co-Scientist. *Nature*, 655(8122): 487–496.

He, Z.; Wang, Y.; Zhi, C.; Hu, Y.; Chen, T.-P.; Yin, L.; Chen, Z.; Wu, T. A.; Ouyang, S.; Wang, Z.; Pei, J.; McAuley, J.; Choi, Y.; and Pentland, S. 2026. MemoryArena: Benchmarking Agent Memory in Interdependent Multi-Session Agentic Tasks. *arXiv preprint arXiv:2602.16313*.

Heck, M.; van Niekerk, C.; Lubis, N.; Geishauser, C.; Lin, H.-C.; Moresi, M.; and Gašić, M. 2020. TripPy: A Triple Copy Strategy for Value Independent Neural Dialog State Tracking. In Pietquin, O.; Muresan, S.; Chen, V.; Kennington, C.; Vandyke, D.; Dethlefs, N.; Inoue, K.; Ekstedt, E.; and Ultes, S., eds., *Proceedings of the 21th Annual Meeting of the Special Interest Group on Discourse and Dialogue*, 35–44. 1st virtual meeting: Association for Computational Linguistics.

Hu, Y.; Wang, Y.; and McAuley, J. 2025. Evaluating memory in llm agents via incremental multi-turn interactions. *arXiv preprint arXiv:2507.05257*.

Instacart. 2017. The Instacart Online Grocery Shopping Dataset 2017. https://www.instacart.com/datasets/grocery-shopping-2017. Accessed 2026-05-25.

Jiang, B.; Yuan, Y.; Shen, M.; Hao, Z.; Xu, Z.; Chen, Z.; Liu, Z.; Vijjini, A. R.; He, J.; Yu, H.; Poovendran, R.; Wornell, G.; Ungar, L.; Roth, D.; Chen, S.; and Taylor, C. J. 2025. PersonaMem-v2: Towards Personalized Intelligence via Learning Implicit User Personas and Agentic Memory.

Kang, J.; Ji, M.; Zhao, Z.; and Bai, T. 2025. Memory OS of AI Agent. In Christodoulopoulos, C.; Chakraborty, T.; Rose, C.; and Peng, V., eds., *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 25961–25970. Suzhou, China: Association for Computational Linguistics. ISBN 979-8-89176-332-6.

Kim, T.; Yoon, H.; Lee, Y.; Kang, P.; and Kim, M. 2022. Mismatch between Multi-turn Dialogue and its Evaluation Metric in Dialogue State Tracking.

Kwa, T.; West, B.; Becker, J.; Deng, A.; Garcia, K.; Hasin, M.; Jawhar, S.; Kinniment, M.; Rush, N.; Arx, S. V.; Bloom, R.; Broadley, T.; Du, H.; Goodrich, B.; Jurkovic, N.; Miles, L. H.; Nix, S.; Lin, T.; Painter, C.; Parikh, N.; Rein, D.; Sato, L. J. K.; Wijk, H.; Ziegler, D. M.; Barnes, E.; and Chan, L. 2026. Measuring AI Ability to Complete Long Software Tasks.

Lee, R. 2024. Credit Card Transaction Dataset.

Li, Y.; Guo, W.; Zhang, L.; Xu, R.; Huang, M.; Liu, H.; Xu, L.; Xu, Y.; and Liu, J. 2026. Locomo-Plus: Beyond-Factual Cognitive Memory Evaluation Framework for LLM Agents.

Lu, C.; Lu, C.; Lange, R. T.; Foerster, J.; Clune, J.; and Ha, D. 2024. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*.

Maharana, A.; Lee, D.-H.; Tulyakov, S.; Bansal, M.; Barbieri, F.; and Fang, Y. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. arXiv:2402.17753.

Microsoft. 2026. STATE-Bench: Stateful Task Agent Evaluation Benchmark. https://github.com/microsoft/STATE-Bench. Blog announcement: https://opensource.

microsoft.com/blog/2026/05/19/introducing-state-bench-a-benchmark-for-ai-agent-memory/; accessed 2026-07-28.

OpenAI. 2024. New embedding models and API updates. https://openai.com/index/new-embedding-models-and-api-updates/. Released 2024-01-25. Accessed 2026-05-25.

Packer, C.; Wooders, S.; Lin, K.; Fang, V.; Patil, S. G.; Stoica, I.; and Gonzalez, J. E. 2024. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.

Park, J. S.; O'Brien, J. C.; Cai, C. J.; Morris, M. R.; Liang, P.; and Bernstein, M. S. 2023. Generative Agents: Interactive Simulacra of Human Behavior.

Qwen Team. 2026. Qwen3.5: Towards Native Multimodal Agents. https://qwen.ai/blog?id=qwen3.5. Accessed 2026-07-28.

Rasmussen, P.; Paliychuk, P.; Beauvais, T.; Ryan, J.; and Chalef, D. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956.

Rastogi, A.; Zang, X.; Sunkara, S.; Gupta, R.; and Khaitan, P. 2020. Towards Scalable Multi-domain Conversational Agents: The Schema-Guided Dialogue Dataset.

Williams, J.; Raux, A.; Ramachandran, D.; and Black, A. 2013. The Dialog State Tracking Challenge. In Eskenazi, M.; Strube, M.; Di Eugenio, B.; and Williams, J. D., eds., *Proceedings of the SIGDIAL 2013 Conference*, 404–413. Metz, France: Association for Computational Linguistics.

Wu, D.; Ji, Z.; Kawatkar, A.; Kwan, B.; Gu, J.-C.; Peng, N.; and Chang, K.-W. 2026a. LongMemEval-V2: Evaluating Long-Term Agent Memory Toward Experienced Colleagues.

Wu, D.; Wang, H.; Yu, W.; Zhang, Y.; Chang, K.-W.; and Yu, D. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. arXiv:2410.10813.

Wu, S. 2024. Introducing Devin, the First AI Software Engineer. Cognition AI Blog. Accessed: 2026-05-26.

Wu, X.; Li, K.; Zhao, Y.; Zhang, L.; Ou, L.; Yin, H.; Zhang, Z.; Yu, X.; Zhang, D.; Jiang, Y.; Xie, P.; Huang, F.; Cheng, M.; Wang, S.; Cheng, H.; and Zhou, J. 2026b. ReSum: Unlocking Long-Horizon Search Intelligence via Context Summarization.

Xu, A.; Lin, B.; Xue, B.; Wang, B.; Xu, B.; Wu, B.; Zhang, B.; Lin, C.; Dong, C.; Ling, C.; et al. 2026. Deepseek-v4: Towards highly efficient million-token context intelligence. *arXiv preprint arXiv:2606.19348*.

Xu, W.; Liang, Z.; Mei, K.; Gao, H.; Tan, J.; and Zhang, Y. 2025. A-MEM: Agentic Memory for LLM Agents. arXiv:2502.12110.

Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering.

Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2024. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains.

Zhao, Y.; Yuan, B.; Huang, J.; Yuan, H.; Yu, Z.; Xu, H.; Hu, L.; Shankarampeta, A.; Huang, Z.; Ni, W.; Tian, Y.; and Zhao, J. 2026. AMA-Bench: Evaluating Long-Horizon Memory for Agentic Applications. arXiv:2602.22769.

A State Drift Labeling and Validation Details

A.1 Buckets and labeling procedure

Each confirmed failure is one $(case, turn, slot)$ point assigned to exactly one bucket. An ordered rule stops at the first match: (Q1) is the gold fact in context? (no $\rightarrow$ retrieval failure) (Q2) is the gold well defined? (no $\rightarrow$ gold issue; $\leq 3\%$ everywhere, omitted from Table 1) (Q3) does the output engage the slot domain? (no $\rightarrow$ comprehension failure) (Q4) is it correct but wrongly formatted? ($\rightarrow$ schema mismatch) (Q4.5) where the gold may be an abstention, did the agent confabulate an in-domain value? ($\rightarrow$ failed abstention; vacuous where gold is always concrete) (Q5) does the output surface the current value but err in the inference? ($\rightarrow$ reasoning failure). Only a point surviving all six reads, whose output matches an available but stale or wrong-scope version of a never-retracted fact, is labeled drift; points whose evidence supports no assignment are dropped, not defaulted. The never-retracted condition excludes legitimate update: the judge must cite evidence that the newer version was still in force, so an agent following a genuine revision is never charged. Q1 is resolved mechanically where possible: in shopping and search the gold token is in the printed candidate set by construction; in travel a prompt scan resolves 99.1% of cases; on LongMemEval oracle availability is guaranteed by the prompt. Q2–Q5 are judge-applied.

Notes on Table 1. Rows reach 100% together with an omitted gold-issue share ($\leq 3\%$). MA rows pool 8 memory variants after the adversarial filter. "FA" is "–" where the benchmark's gold is never an abstention. The LME row reports the cross-family confirmed drift rate with the remaining cells from the conservative adjudication (§A.2); its drift share carries a Wilson 95% CI of $[30\%, 60\%]$ (n=36). The τ^2-Z row is descriptive: drift CI $[39\%, 82\%]$, and its 0.0% cells are indistinguishable from non-observation.

A.2 Conservatism: adversarial filter and cross-family confirmation

The adversarial filter, a second pass that attempts to reclassify every drift candidate as another bucket, downgrades 32–47% of candidates across benchmarks, so surviving labels are conservative by construction. Judging proceeds in two stages. First, two independent decodes of the same DeepSeek-V4 judge (temperature 0, identical decision-tree prompt) label every confirmed failure; their agreement is Table 7. The twelve binary disagreements are structured: GPT-4o pulls seven DeepSeek-drift cases (six of them temporal-reasoning, date-format-adjacent) out to schema mismatch or failed abstention, and pushes four abstention-gold cases into drift; the conservative rule takes the non-drift side of every flip. Second, on LongMemEval oracle (the benchmark carrying the perfect-retrieval claim) we add a judge from a different model family (GPT-4o, same rubric and prompt) and report the *cross-family confirmed* rate: a failure counts as drift only if both families say drift. The DeepSeek pass yields 63.9% drift and GPT-4o 55.6%; the confirmed rate is 44.4% $(16/36)$, which is what the paper reports. Where exactly one family says drift we take the non-drift reading, giving the conservative-adjudicated row in Table 1. Cross-family binary agreement is 69.4% raw (κ=0.37, PABAK=0.39), lower than same-family agreement. This is why we report the confirmed rate rather than either family's rate alone.

Benchmark	n	agr (%)	κ	PABAK	AC1	κ_{mode}
MA-shopping	179	91.6	0.17	0.83	0.91	0.00
MA-search	110	72.7	0.31	0.45	0.55	0.09
LME oracle	36	77.8	0.67	0.56	0.56	0.01
LME oracle (cross)	36	69.4	0.37	0.39	—	—

Table 7: Inter-judge agreement on the binary drift decision: two same-model DeepSeek-V4 decodes (top), and DeepSeek-V4 $\times$ GPT-4o (bottom row cross family). MA-travel is omitted (2 doubly-labeled cases). MA-shopping is the one cell where κ misrepresents reliability: both passes call $>90\%$ of failures drift *pre-filter*, which crushes the chance correction despite 91.6% raw agreement.

A.3 Inter-judge agreement

Cohen's κ deflates mechanically when one class dominates the marginals; PABAK ($= 2P_a - 1$, recomputable from raw agreement alone) and Gwet's AC1 correct for this.

Mode κ (which drift sub-mechanism) is near zero on every benchmark (0.00–0.09) while binary κ is fair-to-substantial: judges agree on *whether* a failure is drift but not on *why the wrong fact won*. This is the empirical basis for §3.3's argument that mechanism must be fixed by construction rather than inferred post hoc.

A.4 Human anchoring

Judge–judge agreement cannot establish that the categories are real, so we anchored the rubric to two human annotators under two protocols. *Track A (structured, n=12)*: annotators answer fixed sub-questions (was the gold fact in context; did the agent reference it within K=3 turns; was the referenced scope correct), then record a free observation from a neutral menu; the bucket is *derived* from these answers rather than chosen, so the taxonomy cannot be imposed by the form. *Track B (blind, n=6)*: annotators describe the failure in free text *before* seeing the taxonomy; a researcher then maps descriptions to buckets post hoc. The sample is stratified across LME-oracle, MA-shopping, and MA-search and oversamples judge-disputed cases ($6/12$ in Track A), making it a deliberately hard audit set.

Inter-annotator reliability. On Track A the annotators agree on the binary drift decision at 83.3% raw (κ=0.43, PABAK=0.67); on the full bucket label, 50.0% raw (κ=0.42). On Track B the neutral surface-observation label reaches κ=0.80 (83.3% raw): blind to the taxonomy, annotators see the same failure surface. All 12 blind free-text descriptions (6 cases $\times$ 2 annotators) map onto existing buckets; no description required a novel category, and one annotator's blind description of an LME case independently reconstructs the salience mechanism ("the agent is just grepping for mentions").

Human vs. judge. Binary agreement with the judge is 58.3% and 41.7% for the two annotators, and both assign

less drift than the judge (8.3% and 25.0% vs. 50.0% on this disputed-enriched sample). The departures are replicated rather than idiosyncratic: on five of twelve cases both annotators converge on the same alternative label, concentrated in two patterns. (i) *Wrong slot vs. drift*: answers returning a person where gold is a date/year, which the judge's *other_drift* mode absorbs and both humans call schema mismatch; the cross-family confirmation rule of §A.2 corrects the same boundary. (ii) *Equivalence*: answers the judge calls schema mismatch that gold's own notes accept as alternate forms (a listed alternate name; an omitted legal suffix), which both humans call not-a-failure. Both patterns are policy boundaries, not disputes about the drift category: the human audit and the cross-family confirmation converge on the same correction, applied before any number is reported. The annotation kit (forms, keys, and the analyzer) is released with the benchmark.

A.5 LongMemEval failure anatomy

Of 306 oracle questions, DeepSeek-V4-Flash fails 36. The conservative-adjudicated distribution is 44.4% state drift (16), 27.8% failed abstention (10), 25.0% schema mismatch (9), 2.8% reasoning (1), 0% comprehension and retrieval (recall = 1.0 by construction). By question type, failures of multi-session questions are drift at 71.4% (10/14) vs. 25.0% (2/8) for knowledge-update, identical under the single-family and cross-family confirmed labelings.

A.6 Reasoning-vs-state separation A/B

If drift were reasoning shortfall in disguise, a stronger reasoner would close the gap. We test this directly on LongMemEval multi-session oracle (n=50 paired items, identical prompts, full history in context so no retrieval confound): the only manipulation is enabling DeepSeek-V4-Flash's native reasoning trace, with the judge held constant across arms.

Arm	correct	rate
thinking off	42/50	84.0%
thinking on	38/50	76.0%
Δ		−8.0 pp

Transition	n
off-fail → on-pass	3
off-pass → on-fail	7
both pass	35
both fail	5

Table 8: Paired reasoning A/B. Reasoning does not close the gap: the −8.0 pp point estimate rests on 3 rescues against 7 new failures, which does not establish a direction (exact McNemar p=0.34). The off-pass → on-fail cell includes the case dissected in Appendix B.

The dispositive contrast is the asymmetry, not the point estimate: reasoning leaves the failure set where it was, while state-maintenance interventions move it (the wrapper lifts the same six backends by +20.9 pp mean over each backend alone on this task and backbone, with the structure share decomposed by the matched control in §6.4). These failures lie in state construction, not reasoning capacity.

B Drift Case

One LongMemEval-oracle case (`46a3abf7`, multi-session), chosen because it is cross-family confirmed and both judge families independently assign not only *drift* but the same mechanism (salience). All sessions are in the prompt; recall is 1.0.

What the context establishes. Across three sessions the user has three tanks on record:

- **May 21**, mid-session, inside a question about plant temperatures: *"I've also been taking care of a small 1-gallon tank that I set up for a friend's kid, which has a few guppies. . . "* Mentioned once, never again.
- **May 23**: *"I have a 5-gallon tank with a solitary betta fish named Finley"*; the same session announces *"I've since set up a new 20-gallon community tank."*
- **May 27**: a full session about the 20-gallon, *"I've finally set up my 20-gallon freshwater community tank, which I've named 'Amazonia' "*, followed by a dozen turns on its fish, food, algae, plants, and driftwood.

The probe (May 30). *"How many tanks do I currently have, including the one I set up for my friend's kid?"* Gold: **3**. Agent: **2**.

Why this is drift and nothing else. The question itself names the quiet fact, and the agent still fails to count it. Every alternative reading is closed: not retrieval (oracle: every session is in the prompt), not comprehension (the agent returns a well-formed count), not schema (a number is a number), and not a legitimate update (no tank was ever given away or retracted). Nothing in the output engages the 1-gallon tank. Nor is it reasoning starvation: in the paired A/B of Appendix A.6 this same case flips— without the reasoning trace the model answers 3 (correct); with deliberation enabled it answers 2. What remains is the signature asymmetry: the 20-gallon tank has a *name* and two sessions of repeated discussion; the 1-gallon tank has one clause inside a question about plant care. The louder fact is fully attended; the quieter one, six days and dozens of turns upstream, silently drops out of the state, even when the probe points straight at it. Both judge families label this *state drift, salience mode*, independently and verbatim. This is the mechanism StateMemBench manufactures by construction: a quiet operative fact, a louder competitor, and a decision that requires the quiet one.

C Benchmark Comparison

Table 2 scores each benchmark against the requirements of state-tracking evaluation. The columns:

- *Multi-session dialogue*: probes span temporally separated conversational sessions, so state must persist across session boundaries rather than within one context episode.
- *Scale*: reported size in each benchmark's native unit (turns, sessions, or tasks).

- *Updates central*: state updates are the measured phenomenon, not one question sub-type among many.
- *Supersession*: correct answers require later-overrides-earlier resolution among multiple recorded values of the same fact.
- *Verifiable gold*: the gold answer derives from an executable or programmatic state rather than from annotation (mapping a system's free-form answer onto that key may still use a judge).
- *Drift scored*: the superseded value is a distinct scored outcome, separating acting on old state from generic error.
- *Anti-update controls*: probes that reward retaining an earlier value against a later, non-authoritative mention, so an always-prefer-the-latest heuristic cannot score well.
- *Paired horizons*: the same probes appear at a short and a long horizon where the *operative update history* grows with length, isolating maintenance from distractor volume.

LoCoMo (2024). Multi-session dialogues averaging roughly 300 turns over as many as 35 sessions, with QA over facts and events. Questions target recall, multi-hop composition, and temporal ordering of *static* facts. No question requires resolving a superseded value, gold is free-text judged, and the stale value carries no separate score. Its adversarial and unanswerable questions reward declining when evidence is absent, which is not a control against spurious updates.

LongMemEval (2025). Multi-session dialogue with *knowledge update* as one of six question types (partial updates; those questions require preferring the later value, which makes supersession partial). Gold is curated free text with no programmatic verification, and the stale value is not a scored outcome. Its abstention questions reward declining on missing evidence rather than retaining a value against a spurious update, so they are not an anti-update control. The S and M variants embed the same 500 questions among more filler sessions ($\sim$40–80 vs. $\sim$500), varying distractor volume while the update history itself is unchanged. These are not paired state horizons.

MemoryAgentBench (2025). Mixes conversational and document-stream inputs, making multi-session dialogue partial. Its FactConsolidation split orders counterfactual edit pairs so that the new value follows the old, which makes updates a measured sub-task with later-overrides-earlier resolution and makes part of the gold programmatically constructed, partial on all three counts. Evaluation is answer accuracy under an LLM judge, and the framework does not separate an outdated answer from any other wrong answer, so the stale value is not a scored outcome. There are no anti-update probes and no paired horizons.

MemoryArena (2026). Multi-session, environment-grounded tasks whose interdependent subtasks are scored by execution against the environment, so gold is verifiable. Its sessions are agentic rather than conversational, which makes the multi-session dialogue requirement partial. The measured phenomenon is task completion under memory load: updates arise incidentally, no supersession chains are constructed, and a failure is recorded as task failure rather than as selection of a stale value.

STATE-Bench (2026). Enterprise task episodes across customer support, travel, and shopping, each a self-contained scenario over a pre-populated database with deterministic assertions defining success. The state at issue is environment state mutated by tool calls, not a tracked dialogue state, and each episode stands alone rather than extending a multi-session history. Completion is database-verified, but the reported evaluation also includes an LLM-judged user-experience score, so verifiability is partial. Its policy checks do supply a restricted anti-update control: the simulated user can press for an action the database does not license, and passing requires the agent not to act on that assertion. This gates whether an action is permitted rather than which value of a fact remains operative, so the control is partial. Supersession chains are not constructed, and the stale value is not a distinct outcome.

StateMemBench (ours). Here updates are the measured phenomenon rather than a sub-type. Every probe is generated from an executable state program, which makes the gold answer itself computed rather than annotated; a judge enters only to map free-form answers onto the closed pool. A deterministic word-boundary matcher decides the 28% of answers that name exactly one pool option verbatim and agrees with the judge on 94.1% of those; the judge maps the rest, which name zero or several options. The superseded value is a distinct closed-pool outcome, so drift is scored rather than folded into generic error. Anti-update probes present a later, non-authoritative mention of a value the agent should retain, penalising an always-prefer-the-latest heuristic. And Set A and Set B present identical probe structure at $\sim$165 and $\sim$600 turns, with the operative update history itself lengthening rather than the surrounding distractor volume, so length is the only thing that varies.

D StateMemBench

D.1 Research DOIs

The following DOIs are of the papers we used to ground our synthetic scenario generation for the research domain. No substantial verbatim text from these papers is reproduced, and we do not make any claims about the papers' factual accuracy. We intend for our generated scenarios and benchmark to be used purely for research purposes.

D.2 StateMemBench: Research Example

This is an **oracle** example scenario from StateMemBench (i.e., we include only turns that are relevant to the probe) from the research domain in the **salience** category. Speakers are named collaborators; the standing rule selects the model with the best F1 as the main-table headline.

Source	DOI
Biomedical (bioRxiv)	10.1101/482679 10.1101/512657
Computer Science (arXiv)	10.48550/arXiv.2407.13248 10.48550/arXiv.2604.10985 10.48550/arXiv.2604.11543 10.48550/arXiv.2604.21253 10.48550/arXiv.2605.02392 10.48550/arXiv.2605.06196

Table 9: Source DOIs used to ground synthetic scenario generation, grouped by domain.

Research / Salience

s1.t1 PI: "One standing call before we dive in: whatever ends up best on F1 goes in the main table, full stop."

s3.t5 Engineer: "I was looking at the classifier numbers this morning: BERT Classifier is at 0.448, Logistic Regression baseline is at 0.338."

s9.t6 Researcher: "Just so it's on the record — BERT Classifier is still what I'm using as the reference point when I'm checking whether the stratified numbers look reasonable. Just the default workhorse for sanity checks."
s12.t4 Researcher: "BERT Classifier is what I'd keep as the reference point while we sort out whether this is a data artifact or a genuine modeling gap."

s15.t1 Researcher: "I was reviewing the latest numbers and, just so it's on the record, Logistic Regression baseline is at 0.695 on F1 now."

Probe (after s15) — PI: "Finalizing the main table this week — which one's the headline row?"
Ground truth: *Logistic Regression baseline* (0.695; it was quietly revised past BERT Classifier, which stayed at 0.448 despite being repeatedly named as the team's "workhorse."). The *drift* answer is BERT Classifier — the salient, most-mentioned model.

D.3 StateMemBench: Shopping Example

This is an **oracle** example scenario from StateMemBench (i.e., we include only turns that are relevant to the probe) from the shopping domain in the **sequence** category. The two-week total is a quantity *derived* from the weekly amount, and must be recomputed after the weekly amount changes rather than re-quoted.

Shopping / Sequence

2.3 user: "Also going forward I want 10 packs of the snack packs each week, just set that as my regular amount."
2.4 assistant: "Noted — 10 packs weekly."

11.7 user: "Snack packs — ok so 10 a week, two weeks, that's 20 packs. So 20."
11.8 assistant: "20 snack packs, done."

15.7 user: "Snack packs — going down to 7 packs a week btw — so 7 today."
15.8 assistant: "Noted. 7 snack packs added."

Probe (after s15) — user: "Placing the 2-week school-trip order — how many packs total?"
Ground truth: *14* (7×2; the weekly amount dropped to 7, so the two-week order recomputes). The *drift* answer is 20 — the earlier two-week total, stated aloud back when the weekly amount was 10 and never recomputed.

D.4 StateMemBench: Personal Finance Example

This is an **oracle** example scenario from StateMemBench (i.e., we include only turns that are relevant to the probe) from the personal finance domain in the **status** category. A tiered standing rule maps the bonus amount to a destination; the probe follows a quarter in which the user emphatically committed to one tier before the amount changed.

Finance / Status

3.1 user: "Setting a rule for when my quarterly bonus lands: if it comes in under 1900, just leave it in checking; if it's between 1900 and 4000, top up the travel sinking fund; between 4000 and 6200 put it toward the car loan; and if it's over 6200, max the Roth contribution."

7.1 user: "Bonus hit. Quarterly came in at $4,883."
7.2 assistant: "Got it. Put it toward the car loan, then."

10.1 user: "The new quarterly bonus just posted — $7,017. I'm maxing the Roth, that's the plan, doing it this week."
10.4 assistant: "Confirmed. I'll treat the Roth as fully funded once that $3,500 clears."

15.3 user: "Quarterly bonus came in, it's $1,517 this time."
15.4 assistant: "Got it."

Probe (after s18) — user: "With this quarter's bonus, I'll max the Roth contribution like last time, right?"
Ground truth: *leave it in checking* (the $1,517 bonus falls in the lowest tier, so the rule re-resolves). The *drift* answer is *max the Roth contribution* — the destination the user loudly committed to for the earlier $7,017 bonus, now stale.

D.5 Scenario Generation Pipeline

Domain Grounding Every sampled program is *grounded* with surface material from real data (data sources in §4.2).

Research We collect open-access research papers and reduce each to a cached *fact sheet* of team roles, methods, datasets, and metrics from which scenario entities are drawn. See Appendix D.1 for paper details.

Shopping We derive consumer personas from the Instacart Online Grocery Basket Analysis Dataset (Instacart 2017), sampling users for diversity over basket size, order cadence, reorder habit, and top department.

Personal Finance We derive personas from the Credit Card Transactions Dataset (Lee 2024), a synthetic credit-card transaction dataset, sampling card holders for diversity over age, income, debt, card count, and credit score.

Rendering A strong LLM (`sonnet-4.6`) renders the symbolic event program into natural multi-session dialogue. The LLM is not given specific turns, but rather general instructions (e.g., "in session 14 the user must state they have received a bonus of $7540"). We also state facts or phrases the LLM must avoid in rendering. Since Set B scenarios are very long, we render in chunks with a running summary provided to the LLM at each new chunk. For shopping and finance, we render scenarios as dialogue between a user and helpful assistant, whereas for research, we render scenarios with multiple named speakers to better simulate memory in collaborative settings. Once scenarios are rendered, we programmatically verify that the load-bearing facts appear, and appear in the correct assigned session, as well as that no banned phrases appear in the final scenario. Any scenarios that fail are re-rendered until they pass.

D.6 Validation

All scenarios in the benchmark pass two filters. First, a strong LLM reader (`sonnet-4.6`), given only relevant sessions, must be able to recover the gold answer on $\geq 2/3$ of samples. Second, a weak LLM reader (`haiku-4.5`) with the full transcript should produce the drift answer instead on $\geq 3/5$ of samples (or the gold for anti-trap).

Two notes on this gate: the answerability threshold is $\geq 2/3$, not 1.0, so absolute scores sit against a ceiling somewhat below 1.0; and difficulty is defined against LLM readers by design, since the benchmark measures whether a memory system protects its reader from such traps. Gold and drift are computed before rendering, so the renderer cannot alter trap semantics.

D.7 Reasoning-vs-State Separation

The reasoning-trace A/B referenced here (holding prompt, context, and model fixed while toggling only the reasoning trace on n=50 paired LongMemEval multi-session oracle cases) is reported in full, with its per-case transition table, in Appendix A (Table 8).

E StateMem

E.1 TurnEncoder

Task Context

```
## Task context:

You are tracking state for a
multi-session user-assistant
conversation. The user states constraints
and preferences; later turns may
supersede or update them. Track:
  - hard constraints
  - soft preferences
  - entity status (active / closed /
superseded)
  - cross-unit dependencies
```

TurnEncoder System Prompt

```
You are a state maintenance system
for a task-executing agent. After each
conversation turn, you update the agent's
tracked state.

## Current conversation state (from prior
turns):
{current_state}

## Latest turn (turn {turn_number}):
{latest_turn}

{task_context}

## Your job:
Analyze the latest turn and determine
what state updates are needed. You can:

1. ADD new units: New constraints,
commitments, or facts that emerged this
turn.
   - From USER statements: explicit
requests, preferences, corrections
   - From TOOL results: new facts
revealed by tool calls (e.g., membership
level, flight status, reservation
details, payment info)
{action_watching_instructions}

2. SUPERSEDE existing units: When new
information contradicts or updates an
existing unit.
   - Example: user changes payment
preference from credit card to gift card
   - Example: tool result shows user is
silver, not gold as claimed earlier
   - Include the ID of the unit being
superseded.
   - Look ACTIVELY for supersession
patterns: "actually", "instead", "no
longer", "switched to", "moved off",
"cancelled", "replaced X with Y", "now Y
not X".

3. TRACK PROGRESS: If the user requested
multiple actions (e.g., "downgrade all 5
reservations", "cancel these 3 flights"),
track what has been completed and what
remains.
```

```
   - Add a progress unit like: "Completed
2 of 5 reservation downgrades. Remaining:
X7BYG1, EQ1G6C, BOH180"
   - Update this unit each turn as more
items are completed.
   - Mark as "hard" priority so the agent
doesn't forget remaining items.

4. LINK CASCADE / DERIVED dependencies:
When a new unit's validity depends on
another unit's status or value, surface
that link.
   - Use `coupled_with: [unit_id]`
when this unit's validity depends
on the STATUS of another unit
(active/closed/superseded).
    Example: "Netflix autopay on Card
3" is coupled_with the unit declaring
Card 3's status -- if Card 3 closes, the
Netflix routing must be re-evaluated.
   - Use `derived_from: [unit_id]` when
this unit's VALUE was computed from
another unit's value.
    Example: "monthly savings $1,800"
is derived_from the unit "take-home
pay $6,000" via a savings-rate rule --
if take-home changes, savings derives
differently.
   - Use `triggers: [{"kind":
"date_passed", "payload": {"date":
"YYYY-MM-DD"}}]` when this unit becomes
operationally relevant on a specific date
or condition.
    Allowed kinds:
"date_passed", "entity_closed",
"supersession_announced", "cascade".

5. NO CHANGE: If the turn contains no
new state-relevant information, or if the
state already captures everything.

## Rules:
- Only create units for information
RELEVANT to future decisions
- Do NOT duplicate information already
in an existing unit -- check the current
state first
- Priority "hard" for: verified
facts, binding commitments, remaining
obligations, things that constrain future
actions
- Priority "soft" for: preferences,
nice-to-haves, things that can yield
- Keep content concise -- one clear
sentence per unit
- Type and scope are free-form labels you
assign
- For coupled_with / derived_from /
triggers: only include when there is a
REAL dependency. Leave empty otherwise.

## Output format:
Return a JSON object:
{
  "add": [
    {
      "content": "string",
      "priority": "hard" | "soft",
      "source_type": "user" | "action" |
"tool_result",
      "type": "string (free-form category
label)",
      "scope": "string (what this applies
to)",
      "coupled_with": ["unit_id_a",
"unit_id_b"],
      "derived_from": ["unit_id_c"],
      "triggers": [{"kind": "date_passed",
"payload": {"date": "2026-06-15"},
"description": "tax deadline"}]
    }
  ],
  "supersede": [
    {
      "unit_id": "string (ID of existing
unit to supersede)",
      "reason": "string (why it's being
superseded)",
      "replacement": {
        "content": "string (new version)",
        "priority": "hard" | "soft",
        "source_type": "user" | "action" |
"tool_result",
        "type": "string",
        "scope": "string",
        "coupled_with": [],
        "derived_from": [],
        "triggers": []
      }
    }
  ]
}

If no updates are needed, return: {"add":
[], "supersede": []}
Output ONLY the JSON object, no other
text.
```

E.2 PolicyEncoder

The PolicyEncoder runs once per scenario over the policy document (if any) to extract binding rules into the same `StateUnit` schema as the TurnEncoder. It plugs in the same **Task Context** block as above. Note that of the evaluated benchmarks, only Tau2 (Barres et al. 2025) actually has specific policy (which should be treated differently from typical evolving state).

PolicyEncoder System Prompt

```
You are a constraint extraction
system. Given a policy document for
a task-executing agent, extract every
rule, constraint, and requirement as a
structured unit.

For each unit, determine:
- content: the rule stated clearly and
completely in one sentence
- priority: "hard" if violating this
would make the task fail or break policy,
```

```
"soft" if it is a preference or guideline
that can yield under pressure
- type: a short free-form label
describing the category (e.g.,
"verification", "payment", "eligibility",
"cancellation", "communication")
- scope: what this rule applies to (e.g.,
"all_reservations", "basic_economy",
"gold_members", "cancellation_requests")

Rules for priority assignment:
- "hard": explicit prohibitions ("must
not", "cannot", "only if"), eligibility
checks, verification requirements, safety
rules, actions the agent must or must not
take
- "soft": guidelines about tone, order
of operations when flexible, optional
offers, preferences

{task_context}

## Policy Document:
{policy}

## Output format:
Return a JSON array of objects. Each
object has:
  "content": string,
  "priority": "hard" | "soft",
  "type": string,
  "scope": string

Extract ALL rules. Do not summarize or
merge rules -- each distinct constraint
gets its own unit. If a paragraph
contains multiple rules, split them.

Output ONLY the JSON array, no other
text.
```

E.3 State Injection (at probe time)

At probe time the agent's system message is assembled with an optional per-scenario `system_prompt` (only used if there is per-scenario policy to encode), followed by the rendered **State Block**, followed by the **Recompute Guidance**. The state block is produced by a deterministic renderer (no LLM); its layout is shown below.

State Injection — Block Layout

```
## BINDING -- THIS CONVERSATION
- [<type>] <unit content>
- [<type>] <unit content>
...
## BINDING -- POLICY RULES
- [<type>] <unit content>
- [<type>] <unit content>
...
## PREFERENCES (yield if necessary)
- [<type>] <unit content>
...
## ! NEEDS RECHECK -- may be stale due to
recent events
- [<type>] <unit content>
  -> trigger: <reason>
- [<type>] <unit content>
  -> trigger: <reason>
...
```

State Injection — Recompute Guidance

```
When answering, work through each
relevant piece of state. Specifically:
 - Hard constraints (medical,
scope-bound) take priority over recent
soft preferences.
 - A unit flagged NEEDS RECHECK is STALE:
an input it was derived
   from has changed, so its baked-in
value is now wrong. Do NOT use
   that number. RECOMPUTE it: take its
stated derivation (e.g.
   'X = A * 2'), substitute the CURRENT
value of each input from the
   active state above, and evaluate. If
the result feeds another
   NEEDS RECHECK unit, recompute that one
from the value you just
   derived -- propagate through the chain
in order before answering.
   Only recompute units on the path to
the question; leave others.
 - For cascade-coupled units, surface the
downstream implication explicitly.
```

F StateMemWrapper

StateMemWrapper carries StateMem's three resolution primitives — supersession precedence, rule-over-instance ranking, and derived-value recomputation — into a single answer-time prompt. State is resolved lazily, scoped to the question, from the turn-numbered transcript: the chronology that decides which of two conflicting values is operative survives because the value chain is extracted *with* its turn ordering, at the moment it is needed. This yields the property the composition sweep exploits: the full StateMem treatment attaches to any backend as a rewrite of the one answer call the backend already makes — no store, no per-turn processing, no added LLM calls, and a call count identical to the backend alone.

F.1 Mechanism

The answer call works in two committed sections. Section 1 (*trace*, capped at 250 words) reconstructs, in chronological order, every turn that establishes, updates, or supersedes the entities the question touches, each cited as `[turn N]`; it must include the standing rules that govern the decision and the most recent stated value of every input those rules apply to, note derived values whose inputs later changed, and end with the current operative value of each relevant entity. The trace is committed before any answer is produced, which forecloses post-hoc rationalization: the model binds itself to a state reading first and answers from it second. Section 2 (*resolve*) commits the answer under four precedence rules — later supersedes earlier; standing rules outrank past instances,

applied to current inputs; derived values are recomputed, never quoted; a fact is retired only by explicit supersession or expiry, and emits one final `ANSWER:` line, which is parsed as the prediction. The full prompt:

StateMemWrapper — Fused Answer Prompt (Trace + Resolve)

```
{system_prompt}

You are answering a question about a long
conversation using state tracing.
## Question: {question}
## Conversation transcript: {transcript}

Work in TWO sections, in order:

## Section 1 -- State trace (under 250
words):
List, in chronological order, every turn
that establishes, updates, or supersedes
the entities the question asks about
([turn N] speaker: what changed). Include
standing rules that govern the decision
AND the most recent stated value of every
input those rules apply to (amounts,
quantities, dates, thresholds) -- even if
mentioned only once or in passing; scan
for them before concluding an input is
unknown. Note derived values whose inputs
later changed. End with the current
operative value of each relevant entity.
Commit to the trace BEFORE answering.

## Section 2 -- Resolution and answer:
Apply, in order:
 (1) later supersedes earlier;
 (2) standing rules outrank past one-off
instances -- apply
   the rule to CURRENT inputs;
 (3) recompute derived values from
current inputs, never
   reuse stale cached numbers;
 (4) a fact is only retired if actually
superseded or expired.

End with exactly one final line:
ANSWER: <the specific value or decision
the question asks for>
```

Wrapper-Ctrl — Matched Control (structure removed)

```
{system_prompt}

You are answering a question about a long
conversation.
## Question: {question}
## Conversation transcript: {transcript}

Work in TWO sections, in order:

## Section 1 -- Relevant information
(under 250 words):
Summarize the information in the
conversation that is relevant to the
question. Commit to this summary BEFORE
answering.

## Section 2 -- Answer:
End with exactly one final line:
ANSWER: <the specific value or decision
the question asks for>
```

F.2 Control

WRAPPER-CTRL is matched on every surface except structure: the same system prompt, the same two-section format, the same 250-word Section 1 budget, the same commit-before-answering instruction, the same final `ANSWER:` line, and the same transcript-plus-chunks context. Its Section 1 asks only to "summarize the information in the conversation that is relevant to the question", no turn numbers, no value chains, no operative-state section, and its Section 2 contains no precedence rules. The wrapper−control delta therefore isolates state structure at equal call count, token budget, and context.

F.3 Composition protocol

A backend attaches through a two-method interface:

```
class MemoryBackend(Protocol):
  def ingest(self, text) -> None: ...
  def search(self, query, k=10) ->
list[str]: ...
```

The backend ingests every turn with its unmodified native pipeline; at the probe, its top-k chunks are appended to the transcript under a framing header matched to the backend type (retrieval excerpts for BM25/Dense, extracted memories for Mem0/A-Mem/LightMem/MemoryOS). The wrapper consumes only strings, so lexical, dense, and LLM-extracted backends satisfy the identical interface, and switching backends is a constructor swap. The transcript is rendered as numbered turns with a 400k-character tail guard so the longest histories fit both backbones' context windows.

F.4 Sweep protocol

Each backend ingests each conversation once; all three answer conditions (alone, +Ctrl, +SMW) are then issued against the same populated store, so condition deltas cannot arise from ingest variance. Case sets are frozen n=60 manifests per benchmark (STATEMEMBENCH: 30 Set-A probes plus 30 Set-B probes sampled from 22 long scenarios—16 contributing one thread-probe, 4 two, 2 all three—at k=10; LongMemEval: multi-session question type, k=20), identical across backends, conditions, and backbones. The frozen set's category mix differs from the full benchmark's (anti-trap is overweighted, status underweighted), so absolute scores in the sweep are mix-dependent and sit above the full-benchmark level; the within-row condition deltas are the quantity of interest. A single fixed deepseek-v4-pro judge scores all conditions in one pass, and every wrapper output is logged with its full state trace, so any scored answer can be audited against the trace that produced it.

F.5 Cost

The wrapper adds no LLM calls and no ingest-time work: it rewrites the single answer call the backend already makes, so call count, storage, and per-turn cost are unchanged, and

overhead is confined to that one call. There it consists of two bounded constants: a fixed instruction block of 155 input tokens — measured constant across both benchmarks and all six backends (360 answers per condition per benchmark) — and a trace capped at 250 words, observed at 169–188 output tokens beyond the matched control. The full state treatment therefore costs ∼350 tokens per question, independent of conversation length, backend, and benchmark: on LongMemEval this is 0.2% of the answer prompt, purchasing the condition deltas of Table 6.

The wrapper does condition on the transcript, so its answer call is transcript-sized (median input 8.6k tokens on StateMemBench, 87k on LongMemEval) where a backend-alone call sees only retrieved chunks. This is the cost of the long-context baseline, not of the wrapper: a long-context answer pays the same input, and the wrapper matches it within 155 tokens while adding the backend's retrieval and the state treatment. The matched control makes the separation exact, control and wrapper differ by 155 input and ∼180 output tokens, so every accuracy delta between them in Table 6 is bought at that price. Median total output, trace and answer included, is 398–416 tokens.

G Results

G.1 Evaluated Baselines

All baselines call the same answer template as the no-memory floor. Whenever we retrieve, we set $k = 10$ for all baselines that take k for StateMemBench and set $k = 20$ for the relatively longer scenarios of LoCoMo and LME. These depths are not a tuned advantage. Sweeping $k \in \{5, 10, 20, 40\}$ for the retrieval baselines (BM25, dense, Mem0) in the alone condition, accuracy on StateMemBench is flat across all depths (e.g. BM25 20.0% at every k; Mem0 25–27%), so k=10 favors no method. On LongMemEval the retrieval baselines instead improve monotonically with k (dense 35.0 → 51.7% and Mem0 26.7 → 45.0% from k=5 to 40; BM25 flat at 16.7%), so k=20 is, if anything, generous to the baselines rather than to StateMem.

Retrieval (BM25, `text-embedding-3-small`). Both ingest the conversation as per-turn chunks and call the LLM only at the answer step. BM25 is lexical retrieval via `rank_bm25` (`BM25Okapi`); the dense retriever uses cosine top-k over OpenAI `text-embedding-3-small` embeddings.

Graph-RAG baselines (nano-graphrag, HippoRAG, LightRAG, GraphRAG). Four graph-structured retrieval systems that build an entity/relation graph over the conversation at ingest and retrieve over it at answer time. nano-graphrag and LightRAG build a local knowledge graph with community summaries; HippoRAG retrieves by personalized PageRank over an OpenIE graph (contriever embedder); GraphRAG (Microsoft) runs its CLI index/query pipeline per scenario. All use the panel's substrate model for extraction and answering, with OpenAI `text-embedding-3-small` for embeddings.

Mem0. Installed via PyPi package `mem0ai` with a ChromaDB vector store and sentence-transformers, using Mem0's default retriever. Mem0 uses a two-stage extraction and update phase, where the LLM can use tool calling during the update phase to keep the memories up to date (Chhikara et al. 2025).

A-Mem. Cloned from `agiresearch/A-mem`, default retriever. We relax its `json_schema` response format to `json_object` for DeepSeek compatibility. A-Mem enables dynamic, agent memory operations; it extracts structured notes, links them, updates linked memories, and retrieves relevant memories at test time (Xu et al. 2025).

LightMem. Cloned from (`zjunlp/LightMem`), default retriever. LightMem focuses on lightweight memory as the name would suggest and has three stages: compression, consolidation, and sleep-time update (Fang et al. 2026).

MemoryOS. Installed via PyPi package `memoryos-pro` (installed `-no-deps` with `faiss-cpu`), default retriever. MemoryOS organizes memory into three tiers (long-, mid-, and short-term) and operates differently and dynamically on different kinds of memory (Kang et al. 2025).

A note on LightMem and MemoryOS scores. Both systems score near floor on every benchmark we run, below any plausible level for published systems. We run the released implementations directly (official clones / PyPI, upstream-example configs; LightMem through its native vLLM provider with topic segmentation and a forced end-of-ingest flush), and roughly 60% of their answers on StateMemBench are still non-answers—little relevant content surfaces at answer time under per-turn conversational ingest. Whether this reflects the methods' sensitivity to this workload or a residual integration gap, the rows are not evidence about best-case capability, so we report them for completeness and exclude them from comparative claims; no headline comparison depends on them.

Per-arm salience floor on Qwen. Supporting §6.3: gold counts on salience probes on the Qwen substrate, Set A (n=33) / Set B (n=18): long-context 5/33 and 0/18; Mem0 1/33 and 1/18; nano-graphrag 5/33 and 0/18; GraphRAG 6/33 and 0/18; StateMem 0/32 and 1/18 (two StateMem Set A probes were dropped at grading, hence 32 rather than 33). Every arm sits at or below 18%, versus 0.569 for GPT-5.4-Nano long-context, so the salience collapse tracks the substrate, not the memory system. On DeepSeek, StateMem's salience is 0.216 (Table 10).

Long-context models (Qwen-3.5-9B, Qwen-3.6-35B-A3B, GPT-5.4-Nano, DeepSeek-V4-Flash). The Qwen models are served locally via vLLM (Qwen-3.5-9B at 131072 token context; Qwen-3.6-35B-A3B reduced to 16384 after KV-cache OOM). Set A scenarios (∼3k tokens) fit the reduced window comfortably, but the longest Set B scenarios (∼7–15k tokens) approach or exceed it once the answer prompt is added and are tail-truncated, so the 35B long-context row should be read as truncation-affected on Set B. (Its Set A score, which no truncation touches, is also below the 9B model's, so truncation is not the sole cause of its weak showing.) GPT-5.4-Nano is queried via the OpenAI API and DeepSeek-V4-Flash via the DeepSeek API; all model

Table 10: State-agreement accuracy on StateMemBench with deepseek-v4-pro judge. Left block: accuracy per horizon and n-weighted overall. Middle block: accuracy by probe category over all 322 probes (status n=116, sequence n=72, salience n=51, anti-trap n=39, compound n=44); Right block: accuracy by domain (finance n=93, shopping n=127, research n=102). Dense retrieval embeds with OpenAI text-embedding-3-small on both substrates; answers come from the panel's substrate model. Per column within each panel, best in **bold**, second best underlined; StateMem ablations excluded from marking.

		By horizon			By category					By domain		
	Condition	Short	Long	Overall	Status	Sequence	Salience	Anti-trap	Compound	Finance	Shopping	Research
Long-context	Qwen-3.5-9B	0.137	0.167	0.149	0.078	0.000	0.098	**0.692**	**0.159**	0.118	0.197	**0.118**
	Qwen-3.6-35B-A3B	0.063	0.146	0.097	0.026	0.000	0.059	0.632	0.023	0.110	0.165	0.000
	GPT-5.4-Nano	**0.311**	**0.227**	**0.277**	**0.172**	**0.153**	**0.569**	0.615	0.114	**0.344**	**0.402**	0.059
	DeepSeek-V4-Flash	0.132	0.174	0.149	0.103	0.056	0.059	0.641	0.091	0.161	0.244	0.020
Qwen-3.5-9B	No memory	0.000	0.015	0.006	0.009	0.000	0.000	0.026	0.000	0.000	0.000	0.020
	BM25	0.095	0.167	0.125	0.034	0.042	0.000	0.820	0.023	0.107	0.102	0.167
	Dense retrieval	0.105	0.167	0.130	0.034	0.014	0.000	**0.923**	0.023	0.118	0.126	0.147
	nano-graphrag	0.126	0.167	0.143	0.035	0.000	0.100	0.895	0.068	0.140	0.149	0.137
	HippoRAG	0.105	0.167	0.130	0.034	0.014	0.000	0.795	**0.136**	0.118	0.126	0.147
	LightRAG	0.089	0.152	0.115	0.026	0.014	0.059	0.769	0.000	0.065	0.126	0.147
	GraphRAG	0.216	**0.235**	0.224	0.164	0.069	**0.118**	**0.923**	**0.136**	0.204	0.197	**0.275**
	Mem0	0.116	0.197	0.149	0.026	0.042	0.039	0.897	0.114	0.151	0.126	0.177
	A-Mem	0.100	0.167	0.127	0.008	0.042	0.000	0.897	0.045	0.118	0.138	0.138
	LightMem	0.021	0.015	0.019	0.017	0.000	0.000	0.102	0.000	0.000	0.016	0.039
	MemoryOS	0.026	0.023	0.025	0.025	0.000	0.000	0.102	0.023	0.022	0.008	0.049
	StateMem (ours)	**0.237**	0.227	**0.233**	**0.181**	**0.208**	0.020	0.846	0.114	**0.269**	**0.205**	0.235
	– *extraction-only*	0.126	0.197	0.155	0.086	0.000	0.078	0.795	0.114	0.161	0.126	0.186
	– + *supersession*	0.137	0.220	0.171	0.112	0.028	0.039	0.872	0.091	0.172	0.165	0.176
	– *w/o dep.-prop.*	0.202	0.250	0.222	0.155	0.139	0.040	0.897	0.140	0.215	0.238	0.207
	– *w/o recompute*	0.195	0.227	0.208	0.190	0.097	0.059	0.846	0.045	0.204	0.213	0.206
deepseek-v4-flash	No memory	0.000	0.008	0.003	0.000	0.000	0.000	0.026	0.000	0.000	0.000	0.010
	BM25	0.105	0.197	0.143	0.034	0.097	0.000	0.872	0.023	0.108	0.110	0.216
	Dense retrieval	0.168	0.258	0.205	0.086	0.181	0.000	**1.000**	0.091	0.194	0.228	0.186
	nano-graphrag	0.174	0.197	0.183	0.095	0.014	0.098	0.949	0.114	0.215	0.173	0.167
	HippoRAG	0.132	0.258	0.184	0.086	0.139	0.000	0.923	0.068	0.151	0.173	0.225
	LightRAG	0.058	0.144	0.093	0.026	0.028	0.020	0.615	0.000	0.065	0.126	0.078
	GraphRAG	0.137	0.227	0.174	0.103	0.042	**0.118**	0.692	**0.182**	0.097	0.126	0.304
	Mem0	0.168	0.189	0.177	0.103	0.028	0.098	0.949	0.023	0.151	0.205	0.167
	A-Mem	0.158	0.258	0.199	0.060	0.264	0.000	0.949	0.023	0.194	0.197	0.206
	LightMem	0.021	0.000	0.012	0.017	0.000	0.000	0.051	0.000	0.000	0.016	0.020
	MemoryOS	0.011	0.045	0.025	0.009	0.000	0.000	0.154	0.023	0.011	0.039	0.020
	StateMem (ours)	**0.411**	**0.295**	**0.363**	**0.302**	**0.347**	**0.216**	0.872	**0.273**	**0.366**	**0.339**	**0.392**
	– *extraction-only*	0.153	0.205	0.174	0.078	0.028	0.157	0.744	0.182	0.215	0.157	0.157
	– + *supersession*	0.337	0.242	0.298	0.241	0.250	0.157	0.846	0.205	0.280	0.283	0.333
	– *w/o dep.-prop.*	0.400	0.333	0.373	0.336	0.375	0.196	0.846	0.250	0.333	0.362	0.422
	– *w/o recompute*	0.326	0.265	0.301	0.267	0.222	0.196	0.897	0.114	0.301	0.244	0.407

reasoning or thinking is disabled, matching the substrate configuration used by the memory baselines. All calls use temperature 0.

H Limitations

StateMemBench uses synthetically-generated data, which limits its ability to genuinely reflect real world use cases for agent memory systems; its traps also come from the same lazy-reader policy family that StateMem is built to counter, so margins on our own benchmark are best read as an upper bound, with the external benchmarks as the generalization check. Additionally, while we show that drift happens even before models and memory systems approach the "true" long horizon, our scenarios ($\sim$3k tokens in Set A, $\sim$7–15k in

Table 11: Accuracy by question type on LongMemEval (top; full set, n=500, k=20) and LoCoMo (bottom; n=1,985; all retrieval k=20), both substrates (thinking off), graded by a fixed deepseek-v4-pro judge. Question-type n in the header. Knowledge-update and temporal-reasoning (LME) and temporal-reasoning (LoCoMo) are the drift-shaped subsets; LoCoMo adversarial rewards abstention. deepseek LME memory-system cells are from the final unified n=500 judge pass (all arms scored together). Per column within each panel, best in **bold**, second best underlined.

			LongMemEval question type					
	Condition	**Overall**	SS-user (70)	SS-asst. (56)	SS-pref. (30)	Multi-sess. (133)	Know.-upd. (78)	Temporal (133)
Qwen-3.5-9B	Long context	0.550	**0.900**	**0.946**	0.333	0.436	**0.744**	0.248
	No memory	0.062	0.086	0.054	0.067	0.053	0.038	0.075
	Mem0	0.566	0.871	0.607	0.533	**0.602**	0.603	0.338
	A-Mem	0.520	0.886	0.929	0.467	0.361	0.679	0.233
	LightMem	0.064	0.086	0.073	0.000	0.053	0.038	0.092
	MemoryOS	0.080	0.114	0.036	0.100	0.060	0.154	0.053
	StateMem (ours)	**0.580**	0.871	0.536	**0.700**	0.519	0.718	**0.398**
deepseek-flash	Long context	**0.666**	**0.957**	**1.000**	**0.600**	0.594	0.782	0.391
	No memory	0.056	0.086	0.018	0.000	0.083	0.038	0.053
	Mem0	0.594	0.929	0.589	0.367	**0.617**	0.692	0.391
	A-Mem	0.510	0.886	0.875	0.333	0.391	0.679	0.218
	LightMem	0.056	0.086	0.018	0.000	0.083	0.038	0.053
	MemoryOS	0.072	0.114	0.000	0.000	0.075	0.179	0.030
	StateMem (ours)	0.656	0.943	0.607	0.400	0.534	**0.795**	**0.624**

			LoCoMo question type				
	Condition	**Overall**	Single-hop (840)	Multi-hop (282)	Temporal (321)	Open-dom. (96)	Adversarial (446)
Qwen-3.5-9B	Long context	**0.612**	**0.865**	0.445	0.412	0.438	0.422
	No memory	0.010	0.008	0.011	0.000	0.042	0.013
	Mem0	0.481	0.637	0.376	**0.595**	**0.448**	0.182
	A-Mem	0.266	0.355	0.131	0.025	0.281	0.357
	LightMem	0.027	0.021	0.039	0.006	0.115	0.027
	MemoryOS	0.040	0.042	0.050	0.009	0.146	0.029
	StateMem (ours)	0.566	0.790	**0.461**	0.296	0.385	**0.442**
deepseek-flash	Long context	0.587	**0.870**	**0.482**	**0.371**	0.479	0.298
	No memory	0.005	0.001	0.004	0.000	0.031	0.009
	Mem0	0.462	0.690	0.401	0.336	0.458	0.161
	A-Mem	0.279	0.379	0.131	0.028	0.219	**0.377**
	LightMem	0.020	0.014	0.032	0.012	0.073	0.018
	MemoryOS	0.031	0.026	0.050	0.009	0.146	0.018
	StateMem (ours)	**0.592**	0.854	0.473	**0.371**	**0.531**	0.354

Set B) are much shorter than the typical modern LLM context window, which limits their ability to truly stress test long horizon state drift.

On the evaluation side, the grading judge shares a model family with one substrate and scenarios are rendered by Claude-family models. Additionally, for cost reasons, both substrates are small-to-mid-scale models, and behavior under frontier answerers is untested. Reported numbers are single runs. Also, the drift-prevalence rates of §3.3 are LLM-judge labels that human annotators tend to undercut. StateMem itself extracts state units at every turn (∼165–600 encoder calls per scenario), so it uses significantly more resources than a long-context LLM on its own. While none of our released data is particularly sensitive, increasing LLM agent capabilities is always worth some consideration. Better state tracking might encourage users to become more reliant on LLM agents, which could have negative effects on users, especially in relatively serious domains like personal finance.

I AI Usage Disclosure

We designed all experiments and analyses ourselves. We used AI coding agents (e.g., Claude Code) to assist in implementing experiment code and to help manage compute infrastructure (e.g., provisioning and running AWS instances). We also

Table 12: **StateMemWrapper sweep on StateMemBench, by probe category and domain** (gold %, frozen n=60 mixed set; category n: status 12, sequence 12, salience 13, anti-trap 13, compound 10; domain n: finance 22, shopping 18, research 20; deepseek-v4-pro judge). Anti-trap probes are controls (correct behaviour is *not* updating), hence high accuracy even for weak arms — and +SMW does *not* inflate there, i.e. it updates selectively rather than aggressively. Per-cell n is 10–13; splits are exploratory. +SMW rows shaded.

			By category					By domain		
	Backend	**Cond.**	Status	Seq.	Sal.	Anti	Comp.	Fin.	Shop.	Res.
Qwen-9B	Mem0	alone	0	8	0	92	10	27	17	20
		+Ctrl	17	17	23	92	0	23	33	40
		+SMW	50	58	77	92	40	64	56	70
	A-Mem	alone	0	0	0	100	10	18	28	20
		+Ctrl	8	25	23	100	10	27	28	50
		+SMW	25	50	62	85	60	55	44	70
	LightMem	alone	0	0	0	15	0	0	0	0
		+Ctrl	8	33	15	100	0	23	22	60
		+SMW	42	58	46	85	40	50	56	70
	MemOS	alone	0	0	0	23	0	5	0	10
		+Ctrl	0	8	31	77	0	23	22	30
		+SMW	42	50	54	92	40	55	56	60
	BM25	alone	0	0	0	92	0	18	17	20
		+Ctrl	0	25	31	85	10	23	28	50
		+SMW	42	58	54	92	30	50	50	70
	Dense	alone	0	0	0	23	0	0	0	10
		+Ctrl	17	25	31	85	10	27	39	50
		+SMW	33	50	46	85	40	50	50	60
deepseek-v4-flash	Mem0	alone	17	8	0	92	20	32	28	20
		+Ctrl	17	50	69	85	60	64	50	50
		+SMW	42	67	77	92	80	73	67	90
	A-Mem	alone	25	17	0	100	20	36	33	20
		+Ctrl	17	33	46	92	70	55	50	50
		+SMW	42	67	92	85	90	86	67	80
	LightMem	alone	0	0	0	8	0	0	0	10
		+Ctrl	8	25	46	77	70	55	44	50
		+SMW	50	67	77	92	80	82	56	80
	MemOS	alone	0	0	0	8	0	5	0	0
		+Ctrl	25	25	54	85	60	55	50	40
		+SMW	58	58	62	92	70	68	56	90
	BM25	alone	0	0	0	92	10	18	22	20
		+Ctrl	25	33	46	85	50	59	33	50
		+SMW	58	50	77	85	80	77	67	80
	Dense	alone	0	0	0	0	0	0	0	0
		+Ctrl	8	33	46	85	60	50	39	50
		+SMW	33	58	69	85	70	68	56	90

used LLMs for polishing writing, suggesting organizational structure for the paper, and generating LaTeX table code. All scientific claims, experimental designs, and conclusions are our own, and the authors take full responsibility for the content of the paper.

Table 13: **StateMemWrapper sweep on LongMemEval: paired outcome transitions vs. the backend alone** (frozen n=60; Dense/Qwen has n=59, one case failed at ingest. The set is drawn from the multi-session question type by construction, so no category split applies). For each backend, each condition's answers are compared case-by-case against the *alone* answers: Fix = alone wrong $\rightarrow$ condition right; Brk = alone right $\rightarrow$ condition wrong; Fix $-$ Brk equals the net case change in Table 6. +SMW is net-positive for every backend on both substrates; the control is net-negative for three Qwen backends (Mem0, A-Mem, Dense).

	Qwen-3.5-9B				**deepseek-v4-flash**			
	+Ctrl		+SMW		+Ctrl		+SMW	
Backend	Fix↑	Brk↓	Fix↑	Brk↓	Fix↑	Brk↓	Fix↑	Brk↓
Mem0	10	11	**18**	12	15	7	15	**6**
A-Mem	7	13	**17**	**8**	**18**	8	16	**4**
LightMem	4	0	**14**	**0**	17	**0**	**20**	1
MemOS	4	3	**10**	**0**	17	2	**20**	2
BM25	7	7	**15**	**3**	16	2	**17**	2
Dense	8	17	**12**	**10**	12	7	**15**	13

Table 14: Closed-pool failure modes on DeepSeek-V4-Flash, n=322 (combined Set A+B; errors are 0 for every method). Other-opt means an answer that was neither the drift target nor the gold truth but *was* in the closed-pool plausible options (constructed at generation time). Off-pool means the method did not answer with a recognizable option (`matched_option` null). Drift-rate is the percentage of answers selecting the drift target over all answers (drift / n).

Method	Correct	**Drift**	Other-opt	Off-pool	Drift-rate
Long-context (V4-Flash)	48	206	4	64	64.0%
Long-context (GPT5.4n)	89	159	1	73	49.4%
StateMem (ours)	**117**	158	11	36	49.1%
extraction-only	56	226	6	34	70.2%
+ *supersession*	96	184	9	33	57.1%
− *dependency-propagation*	120	157	9	36	48.8%
− *recompute-guidance*	97	179	10	36	55.6%
Mem0	57	193	18	54	59.9%
A-Mem	64	181	5	72	56.2%
LightMem	4	48	0	270	14.9%
MemoryOS	8	83	1	230	25.8%
nano-graphrag	59	188	7	68	58.4%
HippoRAG	59	175	12	76	54.3%
LightRAG	30	150	5	137	46.6%
GraphRAG (MS)	56	158	8	100	49.1%
BM25	46	193	5	78	59.9%
Dense	66	177	13	66	55.0%
No memory	1	18	0	303	5.6%

Table 15: Closed-pool failure modes on Qwen-3.5-9B, n=322 (combined Set A+B; two cells n=320 after 2 grading drops, errors otherwise 0). Other-opt means an answer that was neither the drift target nor the gold truth but *was* in the closed-pool plausible options (constructed at generation time). Off-pool means the method did not answer with a recognizable option (`matched_option` null). Drift-rate is the percentage of answers selecting the drift target over all answers (drift / n).

Method	Correct	Drift	Other-opt	Off-pool	Drift-rate
Long-context-9B	48	211	4	59	65.5%
Long-context-35B	31	215	6	68	67.2%
StateMem (ours)	**75**	207	8	32	64.3%
extraction-only	50	239	9	24	74.2%
+ *supersession*	55	230	10	27	71.4%
− *dependency-propagation*	71	212	13	24	66.2%
− *recompute-guidance*	67	220	7	28	68.3%
Mem0	48	202	4	68	62.7%
A-Mem	41	206	4	71	64.0%
LightMem	6	103	0	213	32.0%
MemoryOS	8	112	1	201	34.8%
nano-graphrag	46	191	7	78	59.3%
HippoRAG	42	194	11	75	60.2%
LightRAG	37	151	20	114	46.9%
GraphRAG (MS)	72	185	4	61	57.5%
BM25	40	196	7	79	60.9%
Dense	42	209	17	54	64.9%
No memory	2	78	0	242	24.2%